



slington college
(इस्लिङ्टन कलेज)

Module Code & Module Title

CS6P05NI Project

Interim Report

25% Individual Coursework

Submission: Final Submission

AY 2025 2026

Academic Semester: Autumn Semester 2025

Credit: 30 Credit Year Long Module

Student Name: Rushang Sunuwar

London Met ID: 23050388

College ID: NP01CP4A230444

Assignment Due Date: Tuesday, December 30, 2025

Assignment Submission Date: Tuesday, December 30, 2025

Internal Supervisor: Astha Sharma

External Supervisor: Srawan Shrestha

I confirm that I understand my coursework needs to be submitted online via MST Classroom under the relevant module page before the deadline for my assignment to be accepted and marked. I am fully aware that late submissions will be treated as non-submission and a mark of zero will be awarded.

Table of Contents

Introduction.....	6
1.1. Introduction to the topic.....	6
1.2. Problem Domain	7
1.3. Project as a solution	8
1.4. Aims and Objective	9
1.4.1. Aim 9	
1.4.2. Project Objectives	9
1.5. Report Structure.....	10
1.5.1. Background	10
1.5.2. Development.....	10
1.5.3. Analysis of Progress.....	10
1.5.4. Future Work	10
2. Background and Literature Review.....	11
2.1. Overview of the Target Domain.....	11
2.2. Understanding the Hybrid Approach	12
2.3. Review of Similar Projects	13
2.3.1. Wix AI Website Builder.....	13
2.3.2. v0.dev by Vercel	13
2.3.3. Github Copilot	15
2.4. Comparison Table of Similar Projects.....	16
2.5. Comparative Analysis with VeloSite.....	16
3. Development Methodology.....	18
3.1. Consider Methodologies.....	18
3.1.1. Waterfall Methodologies	18
3.1.2. Pure Agile Methodology.....	19
3.2. Selected Methodology: Agile Incremental Hybrid	21
3.3. Work breakdown Structure	22
3.4. Gantt Chart.....	23
4. Development till Date.....	24
4.1. System Architecture Design	24
4.2. Use case Diagram	25
4.3. Data Flow Diagram	26
4.3.1. DFD Level 0.....	26
4.3.2. DFD Level 1.....	26
4.3.3. DFD Level 2.....	26

4.4.	Sequence Diagram.....	28
4.5.	Communication Diagram	1
4.6.	Activity Diagram	2
4.7.	Flow Chart Diagram	3
5.	Analysis of Progress.....	10
5.1.	Progress Review	10
5.2.	Action Plan	10
5.3.	Progress Table	11
6.	Future work.....	12
6.1.	Remaining task analysis with Gantt Chart.....	12
6.2.	Task Completion According to plan.....	12
7.	Conclusion.....	13
8.	References.....	14
9.	Appendix.....	16
9.1.	Introduction.....	16
9.1.1.	Purpose	16
9.1.2.	Document Conventions.....	16
9.1.3.	Audience and Reading recommendations.	16
9.1.4.	Product Scope	16
9.1.5.	References.....	16
9.2.	Overall Description	17
9.2.1	Product Perspective	17
9.2.2	Product Functions	17
9.2.3	Characteristics and Classes of users.	17
9.2.4.	Operating Environment.....	17
9.2.5.	Constraints of the Design and Implementation.	17
9.3.	External Interface Requirement.	18
9.3.1.	User Interfaces	18
9.3.2	Hardware Interfaces.....	18
9.3.3	Software Interfaces	18
9.3.4	Communications Interfaces	18
9.4.	System Features.....	18
9.4.1	Feature 1 Classifying the intent (AI Component):	18
9.4.2	Feature 2 Content Generation (AI Component).....	18
9.4.3	Feature 3 Deterministic Rendering (System Component).	19
9.5.	Other Nonfunctional Requirements.	19
9.5.1	Performance Requirements	19
9.5.2	Safety Requirements	19
9.5.3	Security Requirements	19

9.5.4 Software Quality Attributes (Fault Tolerance)	19
9.5.5 Business Rules	19
9.6. Appendix A: Glossary	20
9.7. Description on Methodology and its phases	20
9.8. Data Flow Diagram Detail level explanation	21
9.9. Survey response the project.....	22

Table of Figures

Figure 1 Wix AI Website Builder	13
Figure 2 v0.app	14
Figure 3 Screenshot of v0.dev Code view	14
Figure 4 Github Copilot	15
Figure 5 GitHub Copilot suggesting code patterns (Dias, 2023)	15
Figure 6 Waterfall Methodology (Casucian, 2024)	18
Figure 7 Pure Agile methodology	19
Figure 8 Agile Incremental Hybrid Methodology\.....	21
Figure 9 Work Breakdown Structure	22
Figure 10 Gantt Chart	23
Figure 11 System Architecture Design Diagram.....	24
Figure 12 Use Case Diagram	25
Figure 13 Data Flow Diagram Level 0	26
Figure 14 Data Flow Diagram Level 1	26
Figure 15 Data Flow Diagram Level 2	27
Figure 16 Sequence Diagram	28
Figure 17 Communication Diagram	1
Figure 18 Activity Diagram.....	2
Figure 19 Flow Chart Diagram.....	3
Figure 20 Wireframe of Landing page	4
Figure 21 Wireframe of Command Centre.....	5
Figure 22 Wireframe Full page	6
Figure 23 landing page	7
Figure 24 Command Centre page	7

Figure 25 Full preview Page	8
Figure 26 Project Setup Frontend.....	9
Figure 27 Project setup backend	9
Figure 28 Remaining Task.....	12
Figure 29 Survey question 1	22
Figure 30 Survey Question 2	23
Figure 31 Survey question 3	23
Figure 32 Survey Question 4	24
Figure 33 Survey Question 5	24
Figure 34 Survey Question 6	25
Figure 35 Survey Question 7	25
Figure 36 Survey question 8.....	26
Figure 37 Survey question 9.....	26
Figure 38 Survey question 10.....	27
Figure 39 Survey question 11	27
Figure 40 Survey Question 12	28
Figure 41 Survey Question 13xx	28
Figure 42 Survey Question 14	29

Table of table

Table 1 Comparison Table of sample project	16
Table 2 Advantages and Disadvantages of the Waterfall Methodology	19
Table 3 Full Agile Methodology	20
Table 4 Progress Table	11

Introduction

1.1. Introduction to the topic

Web development has gone through various paradigms, first, the manual handcoding in the 1990s, then the content management systems (CMS) in the 2000s and, more recently, the No-Code/Low-Code platforms era (Rokis, 2023). Although these improvements have been made, the entry barrier to development of quality and deployable web applications is still high. Conventional development involves learning full-stack technologies (HTML, CSS, JavaScript, Backend logic) and No-Code Tools, as simple to use, do tend to be feature-bloated and have an extremely high learning curve.

With the recent advent of Large Language Models (LLM) and Generative AI there is a fourth paradigm, Generative Web Development. Such tools as ChatGPT or GitHub Copilot have proven to be able to write code snippets on demand. Nevertheless, such purely generative models have a fundamental weakness of what is termed as hallucination, i.e., the propensity to produce syntactically viable, but functionally invalid or unsound code (Pearce, 2022). Since LLMs are probabilistic (guessing the next token) and not deterministic (applying strict logic), they often have issues in terms of the structural integrity of the code on a global level (Kabir, 2024).

VeloSite is created in reaction to this shortcoming. It is a Hybrid Architecture that fills the divide between the dependability of conventional software engineering and the inventive acceleration of Generative AI. VeloSite also avoids the risks of broken syntax by removing the structural logic (which is processed by a deterministic Python/Flask backend) and the creative content (which is processed by the Google Gemini API) decoupled. This work will confirm that a hybrid Intent-to-Structure model is better than pure Text-to-Code generation in the process of generating efficient professional, responsive websites.

1.2. Problem Domain

Even with the fast development of web technologies, the sphere of web development experiences the problem of speed, reliability and accessibility trilemma now. The particular issues found in this field that require a new solution are:

1. **The Structure hallucination in Generative AI:** Large Language Models (LLM) are not structurally-engineering dependable, whereas they are useful in creative text generation. According to recent studies, LLMs often produce code with security vulnerabilities or that makes references to nonexistent libraries- a condition called hallucination.
2. **The No-Code Complexity and Price:** The classical solutions of the issue of reliability are traditional No-Code platforms (e.g., Wix, Webflow), which bring considerable friction. Moreover, these sites tend to put them into closed ecosystems with recurring monthly payments without the source code ownership rights.
3. **The "Blank Canvas" Paralysis:** Tools which provide the transition between loose will and actual design are lacking. It is common that the user cannot easily map an abstract concept (e.g., a modern portfolio) to the detailed visual parameters used by web frameworks. The present tools expect the user to be the designer whilst the user may wish that the system be a consultant and will interpret intention into valid design choices automatically.

1.3. Project as a solution

Unlike the current solutions where the system is based solely on probabilistic artificialintelligence models to generate structural scaffolding, VeloSite uses Hybrid 80/20 Architectural structure. This solution isolates deterministic structural reasoning and probabilistic creative content, which allowed costlier and predictable building process.

The system alleviates the shortcomings identified by the system through the following mechanisms:

- **Removal of Structural Hallucinations:** With the help of empirically validated set of responsive Jinja2 and Tailwind CSS templates, the system ensures that it builds Document Object Model (DOM) syntactically correctly. The artificial-intelligence engine (Google Gemini) is limited to producing only JSON configuration data and hence structural integrity violations are practically impossible.
- **Design Decisions are Automated:** In order to fight the process of Blank Canvas Paralysis, the architecture uses an Intent Classification Engine. This module makes use of natural-language-processing to read ambiguous user input, automatically choosing an optimal layout archetype (e.g. portfolio, landing page) and creates a consistent color palette and typographic scheme that generates a consistent color palette and typographic scheme without the need of human intervention.
- **Complexity and Cost Reduction:** Made available as a workflow VeloSite democratizes development by providing a Prompt-to-Deploy. Users are given access to the entire source code (HTML/CSS) at once, enjoying complete ownership and eradicating the vendor lock-in that is inherent to the traditional no-code platforms.

1.4. Aims and Objective

1.4.1. Aim

This project focuses its main goal on designing, implementing, and evaluating a fault-tolerant web-generation system, VeloSite, which uses an 80/20 hybrid architecture to generating natural-language prompts into fully functional, responsive, and production-ready HTML/CSS sites, and is planned to be deployed in December 2025.

1.4.2. Project Objectives

To accomplish the given purpose, the SMART objectives described below have been formulated:

- **Specific (Architecture):** Within Increment 1, build a modular Python (Flask) backend which manages to decouple structural logic and AI content to make sure that the final generated code has zero syntax errors.
- **Measurable (Templates):** Using Tailwind CSS by Increment 2, the library of three different, responsive templates (Portfolio, Blog, and Landing Page) will be built which have a 90 or higher mobile-responsiveness score on Google Lighthouse.
- **Achievable (AI Integration):** Integrate the Increment 3 of the Google Gemini API to reach 95 percent accuracy in the creation of structured JSON payloads to classify the intent and synthesize the content.
- **Relevant (Fault-Tolerance):** Increment 4: Implement a validating gate based on JSON-schema which prevents 100% of possible system failures, which happens as the malformed AI data is automatically replaced by pre-validated fallback assets.
- **Time-Based (Validation):** By the end of 2025, the Testing (User Acceptance): We should conduct User Acceptance Testing (UAT) with 5 non-technical users by making sure that a full site source-code bundle can be generated and exported in less than 60 seconds.

1.5. Report Structure

1.5.1. Background

The background section is where the history of the web development paradigms and the emergence of AI tools are explained. It examines the existing market environment, such as the constraints of currently available No-Code solutions and raw LLMs, which form the basis of why a hybrid solution is needed.

1.5.2. Development

The section describes the technical implementation of VeloSite. It includes the system architecture and database design, and the combination of the Flask backend and the Content Engine. It has technical illustrations and snapshots of what has already been accomplished.

1.5.3. Analysis of Progress

The section analyses how the project has been in accordance to the suggested timeline (Gantt Chart) and Scrum approach. It checks the successful execution of the initial sprints, such as the installation of the core of the backend and the template engine with the basic one.

1.5.4. Future Work

This part presents the further development activities to be performed to make the system complete, such as advanced UI polish, the introduction of picture-based analysis, and, finally, rigorous testing before the system can be deployed.

2. Background and Literature Review

2.1. Overview of the Target Domain

The area of focus of this project (VeloSite) is at the intersection point of Generative Artificial Intelligence and **Rapid Web Application Development** (RWAD). Recently, the web development has been a high demand, with trade-offs between control (manual code) and convenience (template builders).

In the present time (2024-2025), there is a paradigm shift in this field, aided by Transformer-based Large Language Models (LLMs). The industry has shifted the emphasis to the writing of syntax to the declaring intent. This idea is commonly known as Prompt Engineering as Code, and it implies that natural language is now the new software engineering abstraction layer (White, 2023).

Nevertheless, this field is now flooded with so-called Copilot-type tools that can help but cannot be used instead of technical expertise. The new sub-domain of the Autonomous Web Generation seeks to make the human purely optional in the coding chain. **VeloSite** is one such product that serves the so-called Prosumer market of users that need professional-level output (Semantic HTML, CSS Variables, Responsive Layouts) but do not have the technical skills to coordinate full-stack structures by hand.

2.2. Understanding the Hybrid Approach

VeloSite is a web-based software that aims at streamlining the refined process of web site creation using simple inputs in natural language. The answer is a clean and userfriendly front-end interface in which users will have the ability to explain their unique needs, including, “a dark-themed portfolio of a photographer,” and an efficient backend system that will process these queries utilizing Artificial Intelligence. The main part of the solution is a smooth combination of a Python Flask server and the Google Gemini API that complement each other by bridging the gap between abstract human intent and concrete web development requirements.

To guarantee full user experience, the solution will entail other necessary functions like real-time preview mode and the direct source code export option. One can immediately preview the created site in a browser-like frame in order to confirm the design and then download the complete, clean HTML code to be deployed. In general, VeloSite offers a more convenient and centralized system where non-technical users can create professionally-grade websites within minutes, which is adequate in addressing issues on complexity and time wastage experienced with the available systems.

2.3. Review of Similar Projects

The vital part of the project is also to be familiar and study about the existing platforms to solve the problem if automated we generation. Analyzing real-world applications, their functionalities, their offerings and their limitations provide valuable insight that helps us to shape up the project.

2.3.1. Wix AI Website Builder

The Wix AI Website Builder is an upgrade to the Wix ADI (Artificial Design

Intelligence), which was no longer present. It is the latest standard of the industry in terms of No-Code automation. It incorporates a conversational interface in contrast to traditional drag-and-drop editors where elements must be manually positioned. Instead, the user will have a dialogue with an AI agent about what their business aims to do and their aesthetic preferences and then the system will create a fully functional site with context-aware copy, images, and layout (Wix.com, 2025).

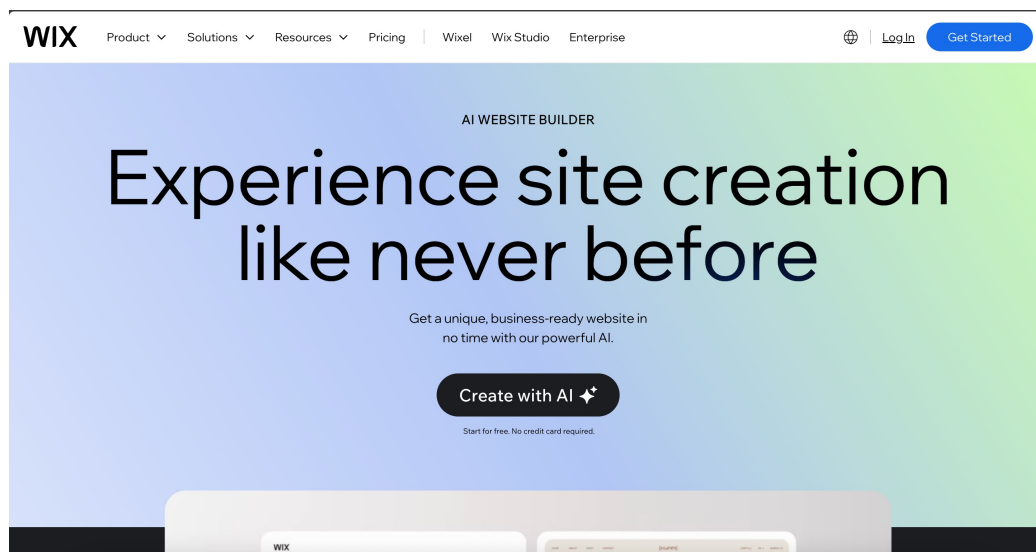


Figure 1 Wix AI Website Builder

2.3.2. v0.dev by Vercel

v0.dev is the future of Generative UI with professional software engineers. It was created by Vercel and uses the state-of-the-art Large Language Models to create User Interface elements, using text input prompts. In contrast to common chatbots, v0 is

directly trained to work with up-to-date web platforms, and generates code ready for production in React and Tailwind CSS (that is, the shadcn/ui library).

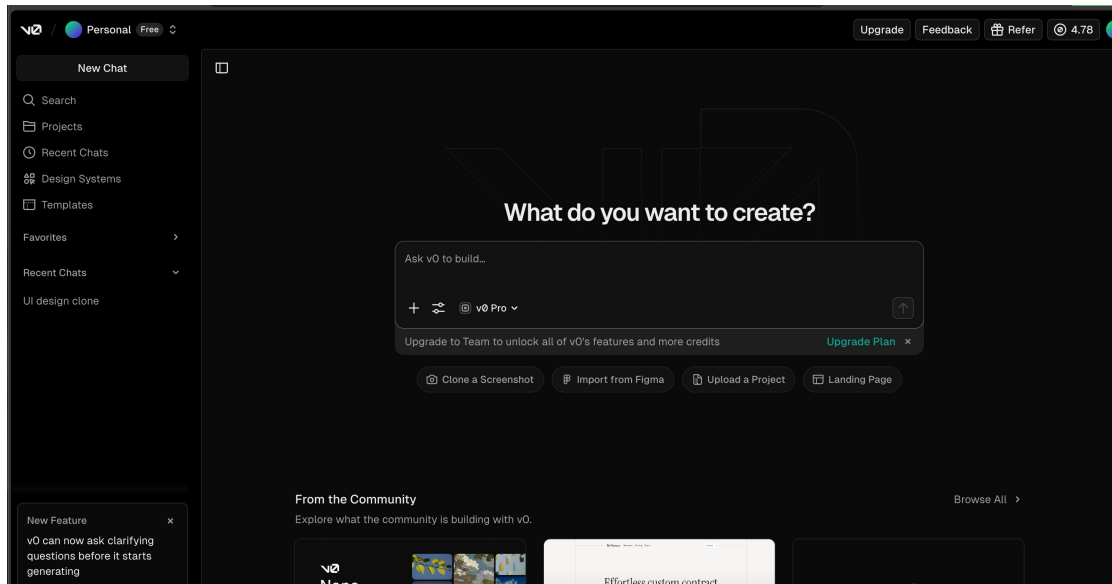


Figure 2 v0.app

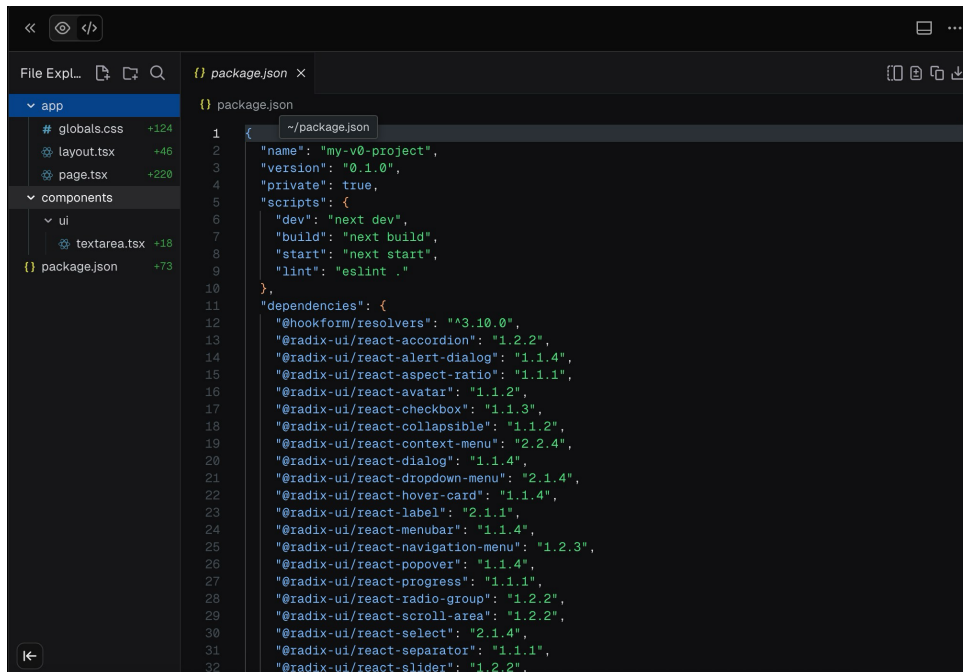


Figure 3 Screenshot of v0.dev Code view

2.3.3. Github Copilot

The market leader and the most widely used AI Pair Programmer is GitHub Copilot created by GitHub in partnership with OpenAI. It is directly embedded in Integrated Development Environments (IDEs) such as VS code to propose code lines or whole functions according to their context. It is the "Assistant" model of AI implementation, in which the AI is used to speed up the process of a human developer.

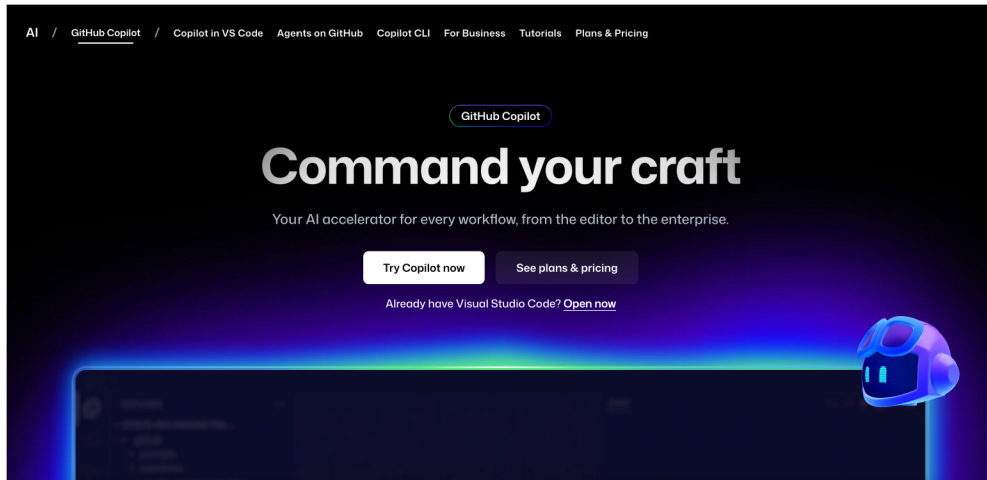


Figure 4 Github Copilot

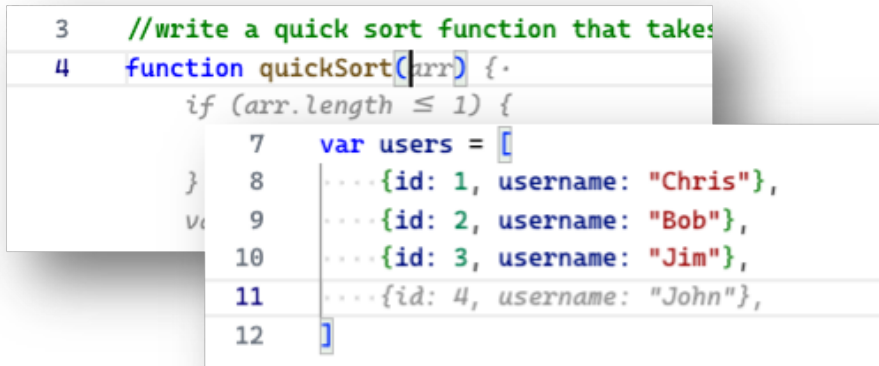


Figure 5 GitHub Copilot suggesting code patterns (Dias, 2023)

2.4. Comparison Table of Similar Projects

Feature /Requirement	Wix AI Builder	v0.dev (Vercel)	GitHub Copilot	VeloSite (Proposed)
Generates Full Websites	Yes	No (Components)	No(Snippets)	Yes
No Technical Knowledge Req.	Yes	No	No	Yes
100% Code Ownership	No	Yes	Yes	Yes
Free / Open Source	No	Partial	No	Yes
Integrated Hosting/Domain	Yes	No	No	No
Complex Backend Logic (DB)	Yes	No	Yes (If coded)	No
Infinite Layout Freedom	Yes	Yes	Yes	No (Template Bound)
Zero Setup	Yes	No	No	Yes

Table 1 Comparison Table of sample project

2.5. Comparative Analysis with VeloSite

The existing market is polarized around either accessible "Walled Gardens" (such as Wix) which do not allow the ownership of the code, or complicated "Developer Tools" (such as v0.dev) which demand substantial technical knowledge. VeloSite is in a good position to bridge this gap. It is a mix of simplicity provided by Wix and the openness of developer environments, providing 100% Code Ownership and Zero Setup to make professional web creation accessible to users who are not technical.

VeloSite however makes special architectural trade-offs to ensure structural integrity. As opposed to Wix, it lacks built-in hosting or database management and is just a pure static code generator. Moreover, as opposed to unlimited freedom of GitHub Copilot, VeloSite is template-constrained. This is an artificial constraint: limiting the AI to a so-called Curated Menu of layouts that have been pre-tested and can be reused, the system avoids syntax errors and offers mobile responsiveness, and is in effect a tradeoff between reliability and novelty.

3. Development Methodology

3.1. Consider Methodologies

Software development life cycle (SDLC) also represents a very important factor that defines the success of a final year project. Before the finalization of an approach, standard methodologies were systematically considered to find out which best fit would be applied during the experimental implementation of the Generative AI in a deterministic web framework.

3.1.1. Waterfall Methodologies

The waterfall methodology is a structured work approach that guides project manager on a liner path. It has five different work phases starting from requirement, design, implementation, verification and maintaince at the end with each phases having each set of task (Day, 2025).

Phases of Waterfall Methodology



Figure 6 Waterfall Methodology (Casucian, 2024)

S.N.	Reason for Consideration	Reason for Not Picking
1.	It gives an academic timetable a clear, strict framework with a set of distinct milestones thus making it easy to manage	It is dogmatically fixed, once a phase has been reached, it becomes hard to revisit and amend artifacts without spending considerable time.
2.	The requirements are detailed at the beginning, which provides a yielded roadmap of the lifecycle of development.	The use of Generative AI (Gemini) creates a lot of uncertainty; trial and error is the basis of instant engineering, which the linear quality of the Waterfall model cannot allow.
3.	It is a focus on thorough documentation on each stage.	The testing will be carried out at the very end; in case the AI integration does not work in the later phases, the whole project schedule will be to fail.

Table 2 Advantages and Disadvantages of the Waterfall Methodology

3.1.2. Pure Agile Methodology

The Agile methodology is a collaborate way of managing the flow of the project without breaking them into smaller phases which is iteration (Paliwal, 2025).



Figure 7 Pure Agile methodology

S.N.	Reason for Consideration	Reason for Not Picking
1.	It is very dynamic and embraces adaptation of necessities thus conforming to the experimental aspect of AI prompting.	In the absence of a strict architectural scheme, Agile in its pure form can lead to scope creep, allowing the project to become larger than the rigid university deadline.
2.	It enables testing to be performed constantly and thus the bugs can be detected at an early stage of the development cycle.	Agile tends to neglect detailed documentation in support of running software, and as a result, it is inconsistent with the scholarly need of a detailed technical report.
3.	It breaks the project into small manageable sprints.	The assumption that leads to using pure Agile is a necessity of continuously getting feedbacks on the stakeholders; since the project is a personal student work, it should be provided with a more detailed roadmap to be able to complete the Skeleton (Backend) first and the Brain (AI) afterwards.

Table 3 Full Agile Methodology

3.2. Selected Methodology: Agile Incremental Hybrid

To balance between the organizational rigor of the Waterfall/Incremental model and the flexibility in Agile, this project uses Agile Incremental Hybrid Approach. The selected methodology allows the system to be built in functional “Increments) (Modules) and refines the code by an iterative transformation of the code using the so-called “Agile Sprints.”

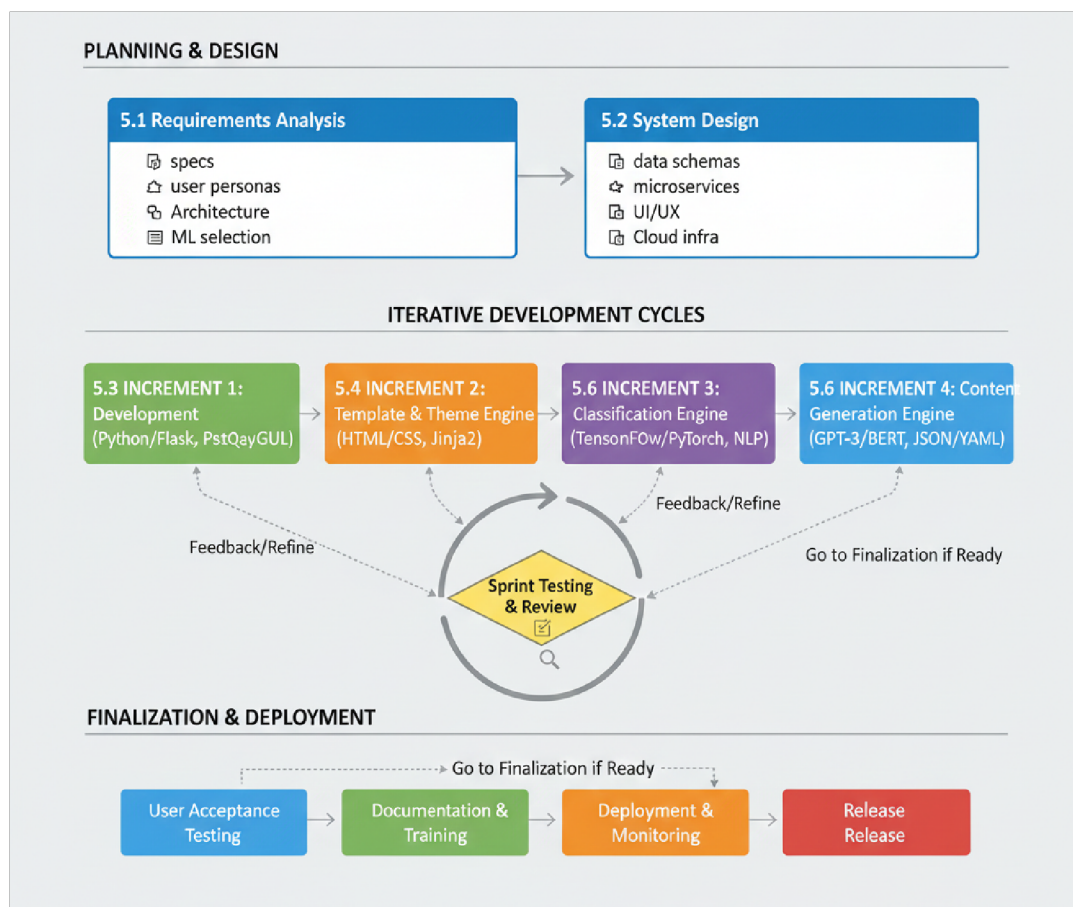


Figure 8 Agile Incremental Hybrid Methodology\

The given method reduces the threat of AI Hallucination since it ensures that the core application is fully operational and devoid of bugs before the introduction of probabilistic AI elements.

3.3. Work breakdown Structure

The work within the projects was broken down in a hierarchical manner in order to make it manageable. Work Breakdown Structure (WBS) is based on the Hybrid structure but splits the project into various requirements, consisting of “Planning”, and finally, the individual increments included in the list above.

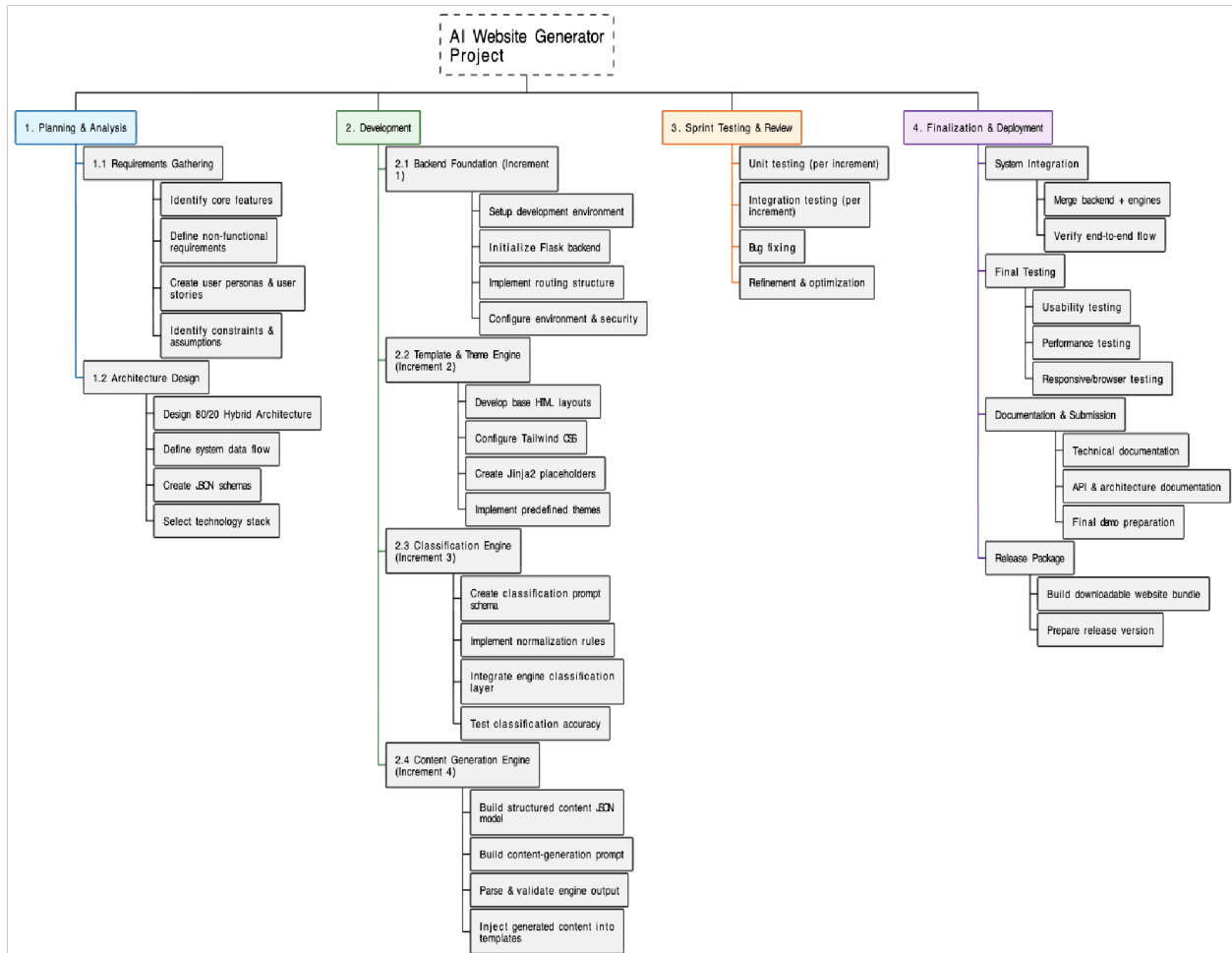


Figure 9 Work Breakdown Structure

[Click Here](#) for clear view of Gantt Chart and WBS

3.4. Gantt Chart

The project schedule was plotted on a Gantt Chart to ensure that the project would be delivered within the stipulated time. The timeline follows the milestones beginning with the first one of the setting the foundation and the of the final polish and MVP delivery.



Figure 10 Gantt Chart

[Click Here](#) for clear view of Gantt Chart and WBS

4. Development till Date

4.1. System Architecture Design

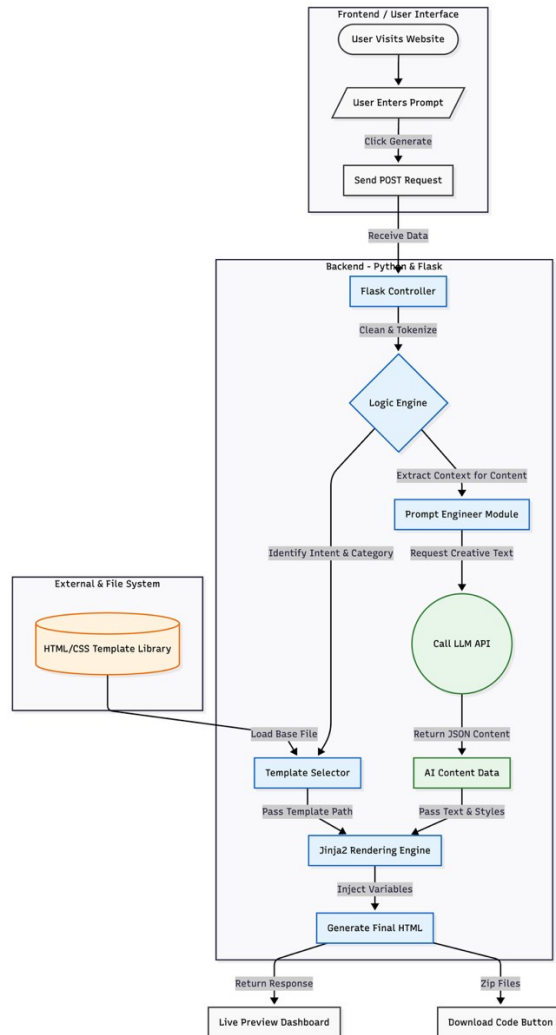


Figure 11 System Architecture Design Diagram

4.2. Use case Diagram

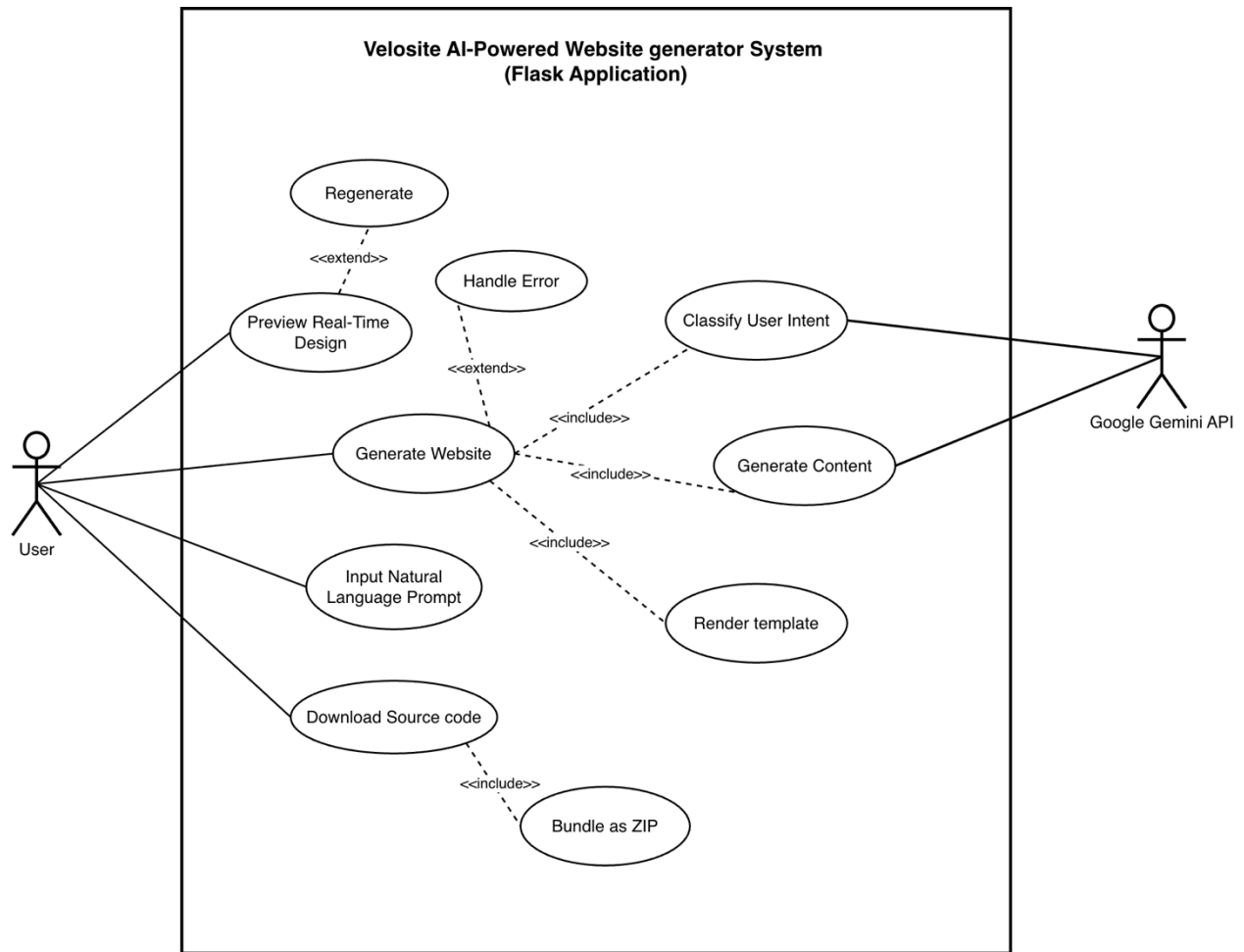


Figure 12 Use Case Diagram

4.3. Data Flow Diagram

4.3.1. DFD Level 0

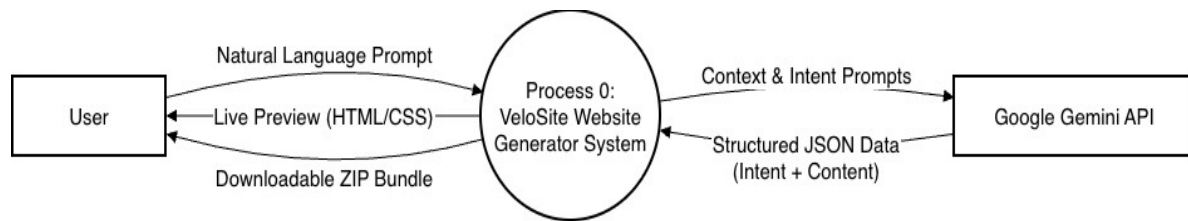


Figure 13 Data Flow Diagram Level 0

4.3.2. DFD Level 1

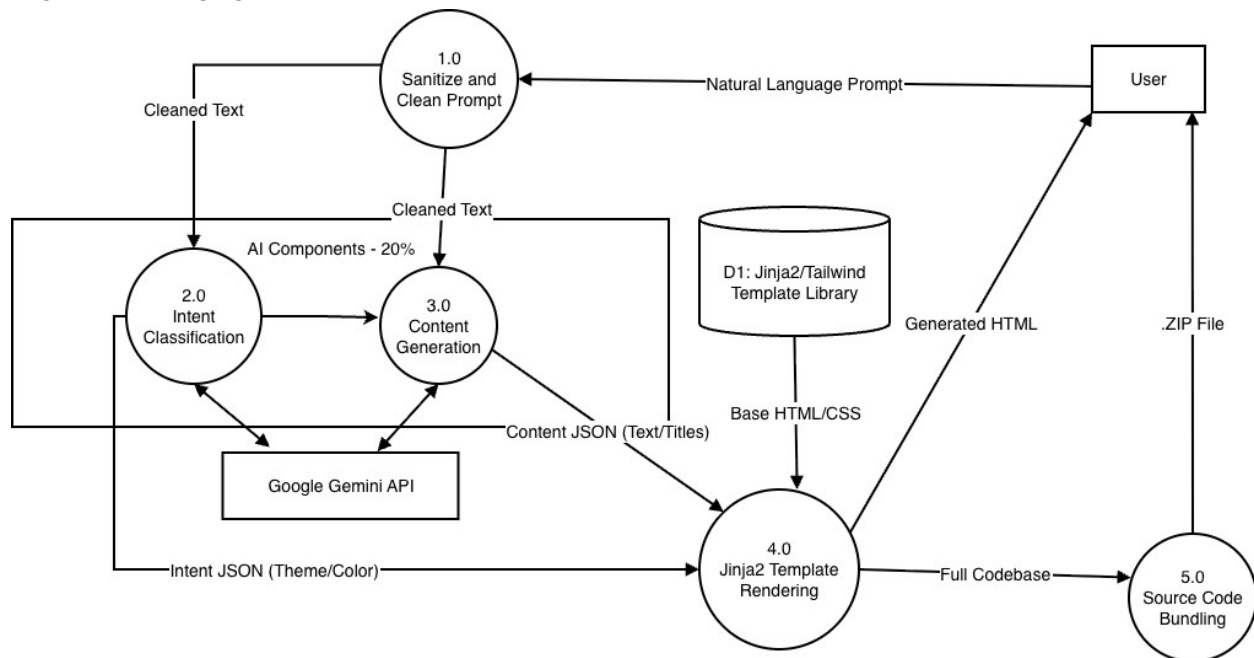


Figure 14 Data Flow Diagram Level 1

4.3.3. DFD Level 2

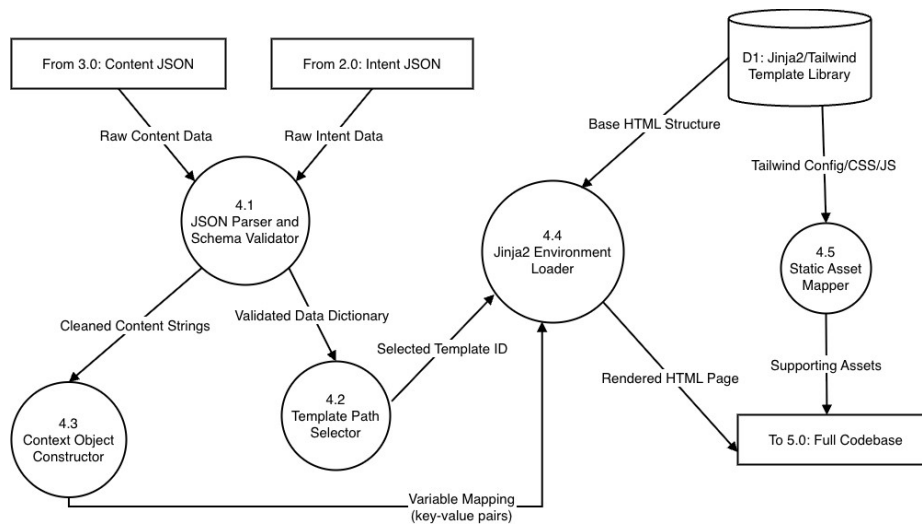


Figure 15 Data Flow Diagram Level 2

4.4. Sequence Diagram

UML sequence diagrams capture the interactions of a collaboration that realizes a use case or operation (instance or generic diagrams), or high level interactions between the users of the system and the system itself, between the system and other systems, or between subsystem components (sometimes called system sequence diagrams) (Visaul Paradigm, 2025).

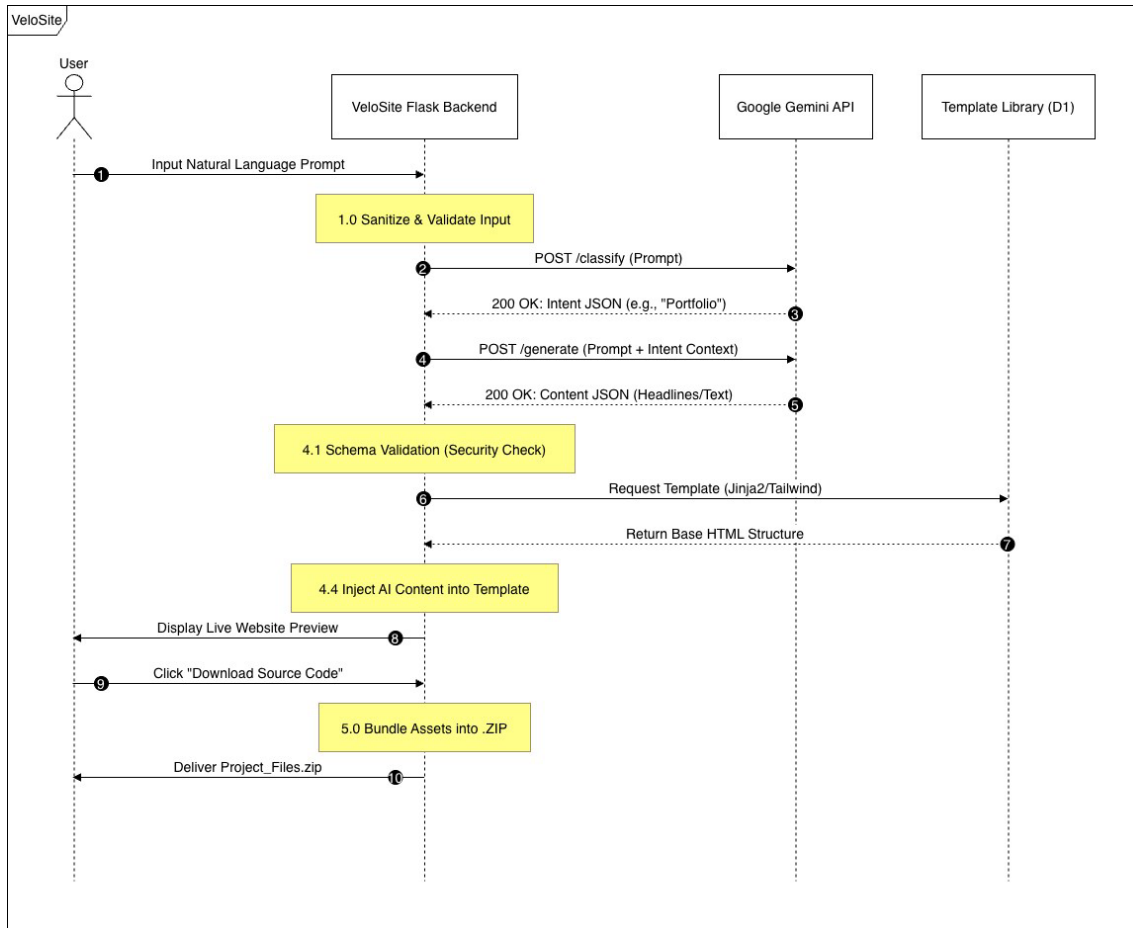


Figure 16 Sequence Diagram

Figure 16 illustrates a sequence diagram of the chronological sequence of interactions between the User and VeloSite Flask backend, as well as Google Gemini API.

4.5. Communication Diagram

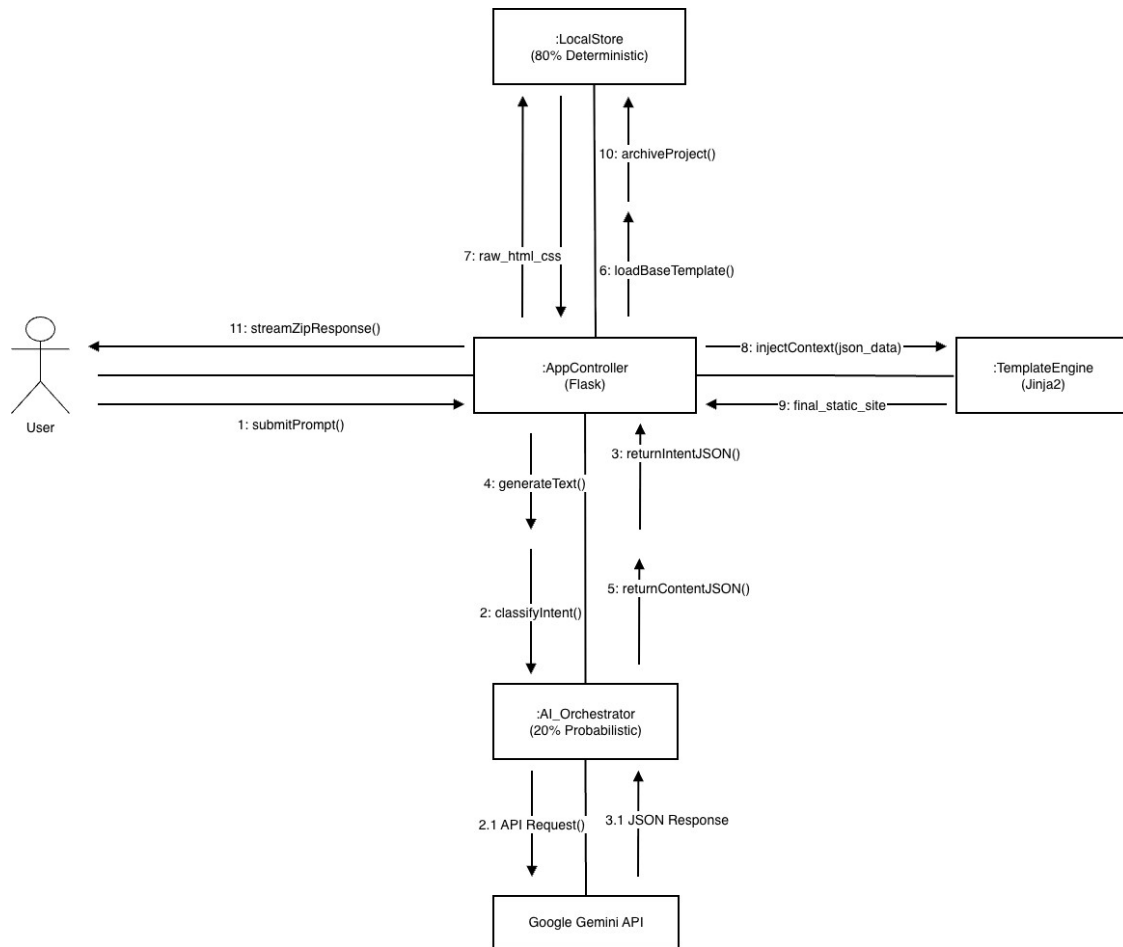


Figure 17 Communication Diagram

Figure 17 Communication Diagram is the structural representation of the VeloSite ecosystem where the focus on the interactions between decoupled parts interacts with each other is emphasized..

4.6. Activity Diagram

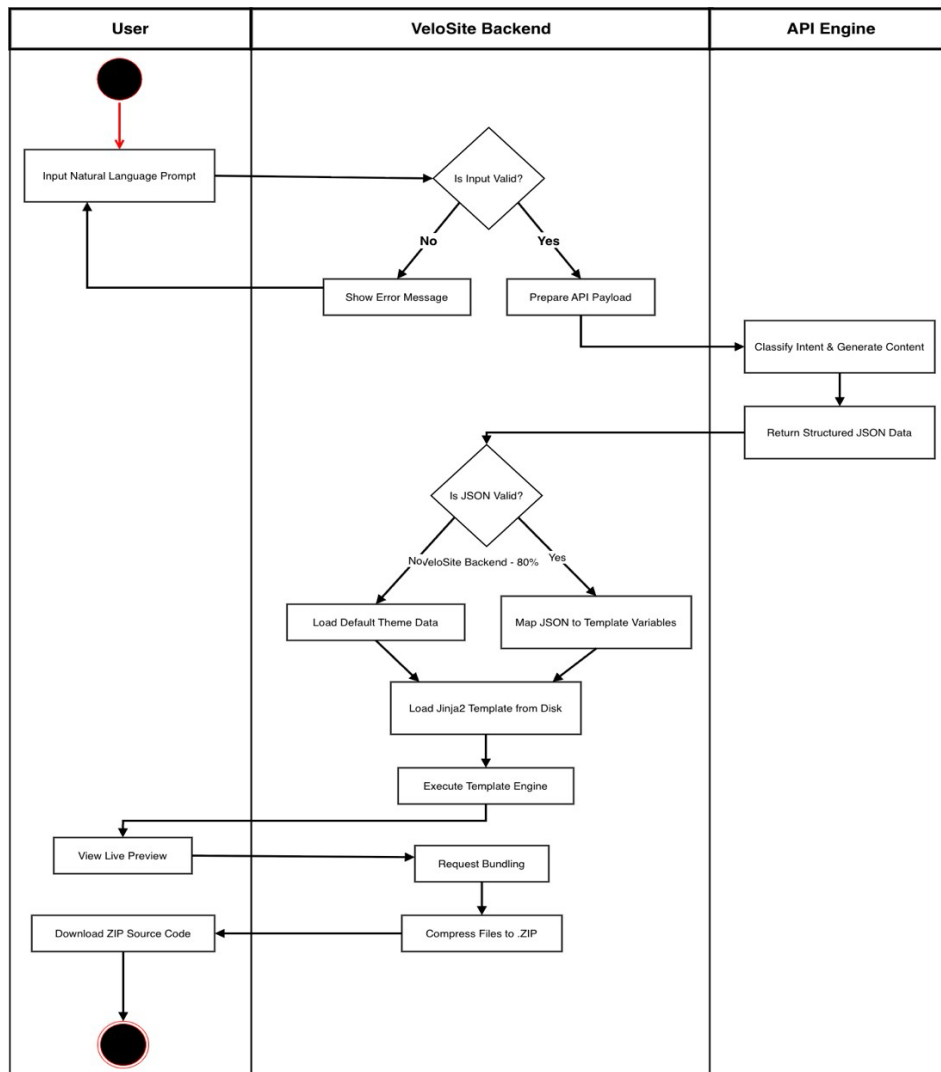


Figure 18 Activity Diagram

Figure 18 shows the Activity Diagram that offers a procedural view of the algorithm of generating websites.

4.7. Flow Chart Diagram

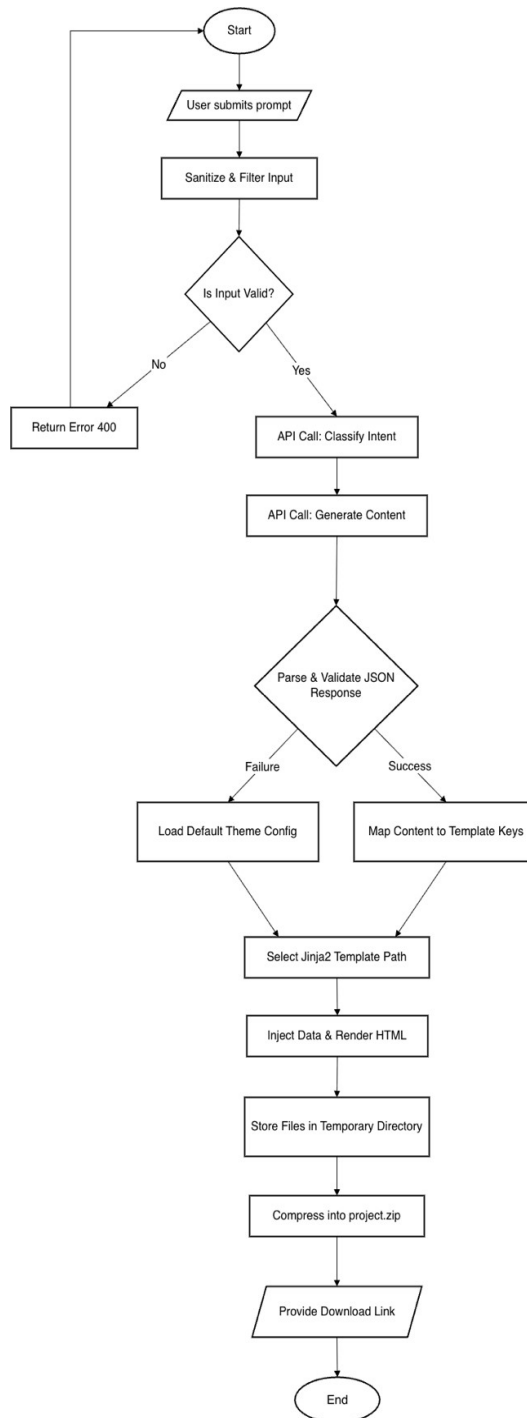


Figure 19 Flow Chart Diagram

Figure 19 is a Flow Chart that illustrates the sequential logic that is followed by the Flask backend when generating the lifecycle.

4.8. Wireframe

4.8.1. Landing page wireframe

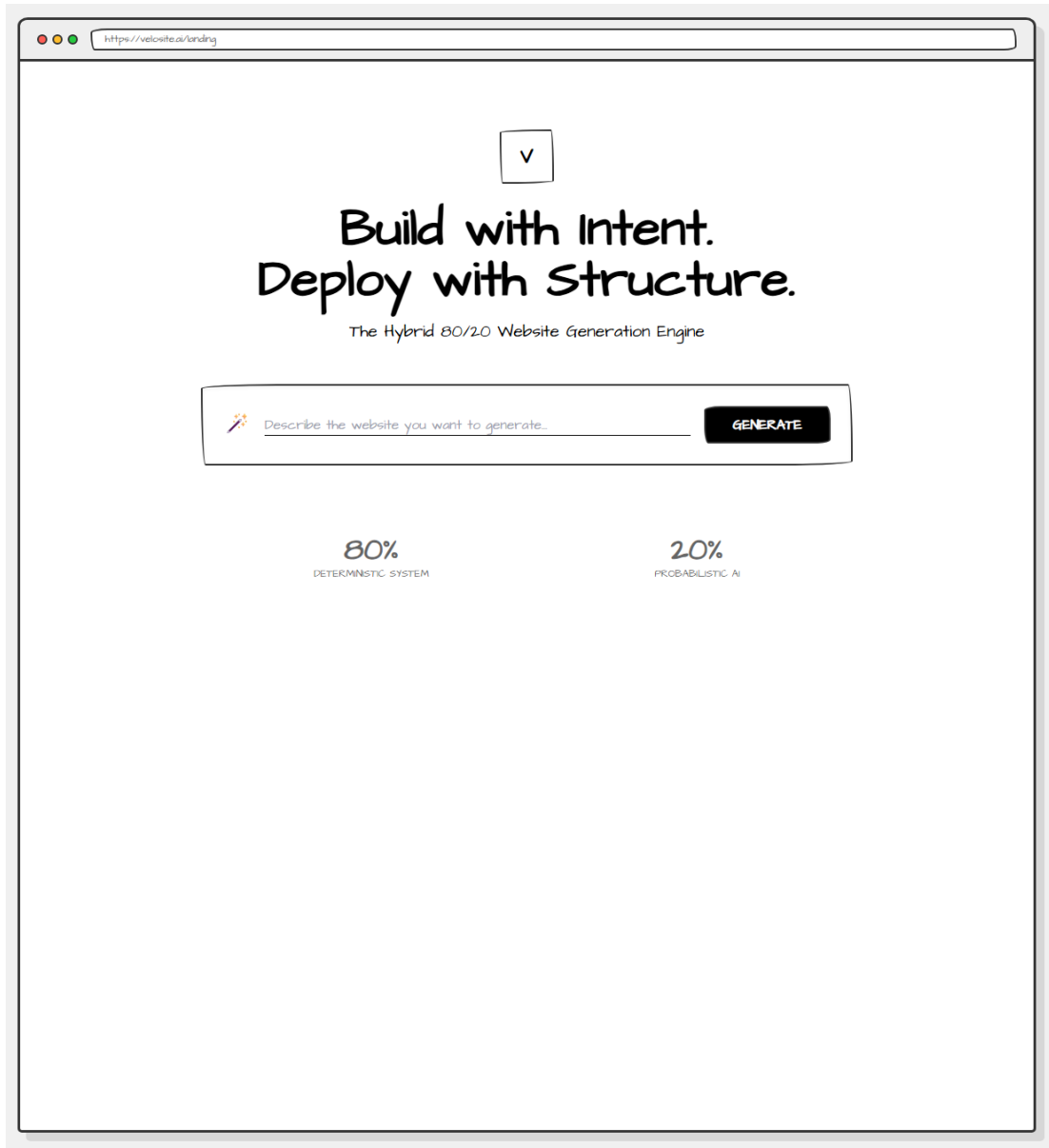


Figure 20 Wireframe of Landing page

4.8.2. Command Centre page wireframe

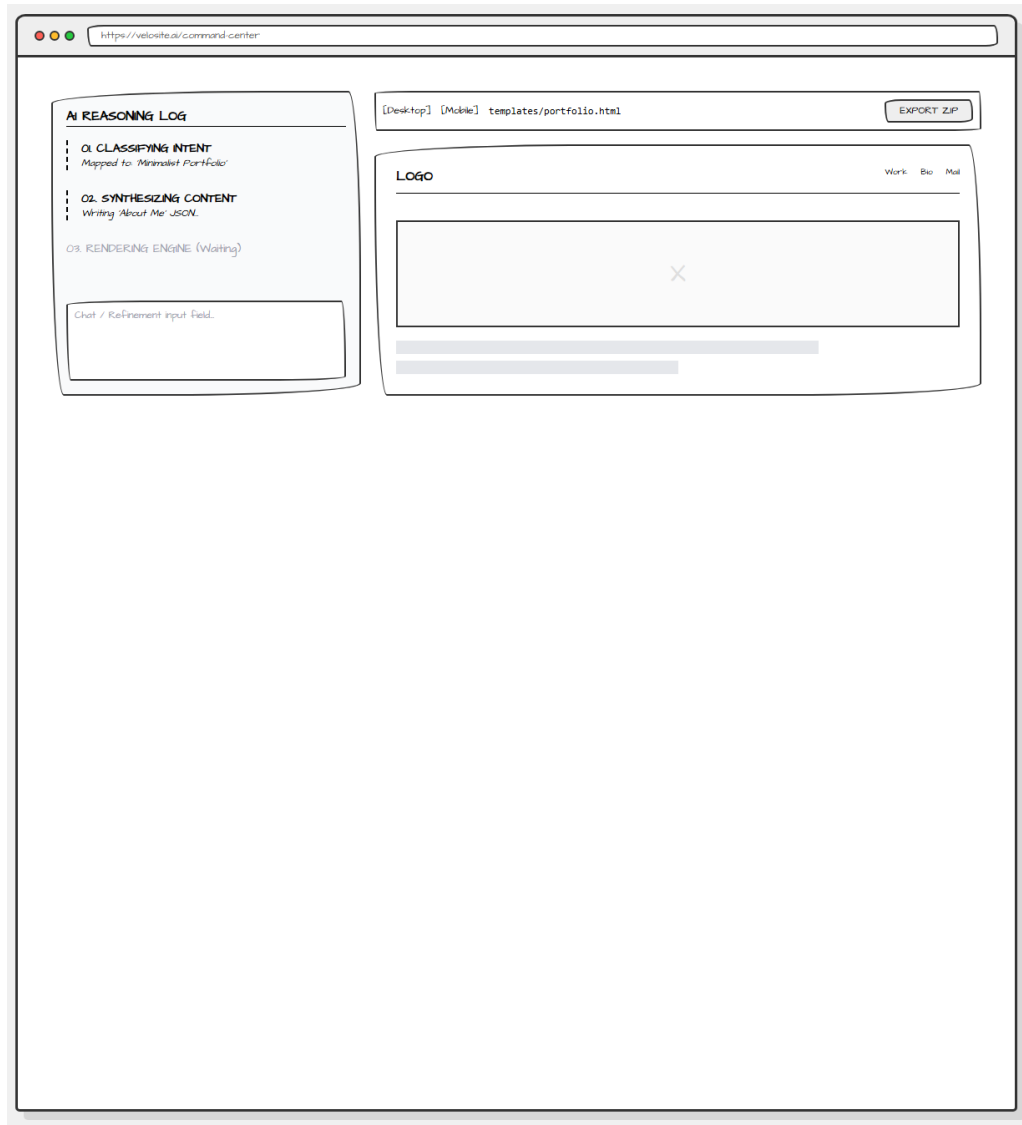


Figure 21 Wireframe of Command Centre

4.8.3. Full preview wireframe

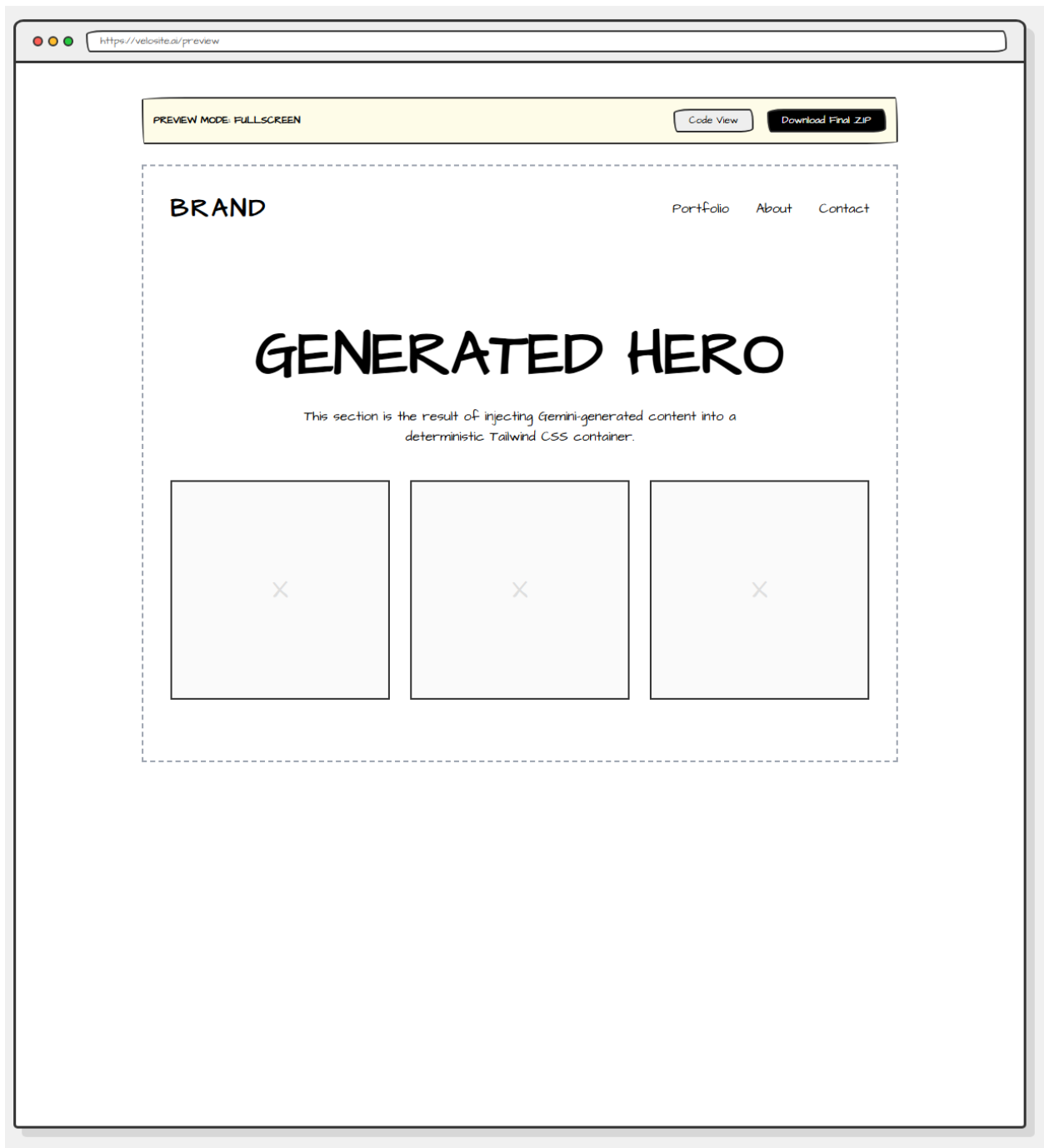


Figure 22 Wireframe Full page

4.9. User Interface UI

4.9.1. Landing page

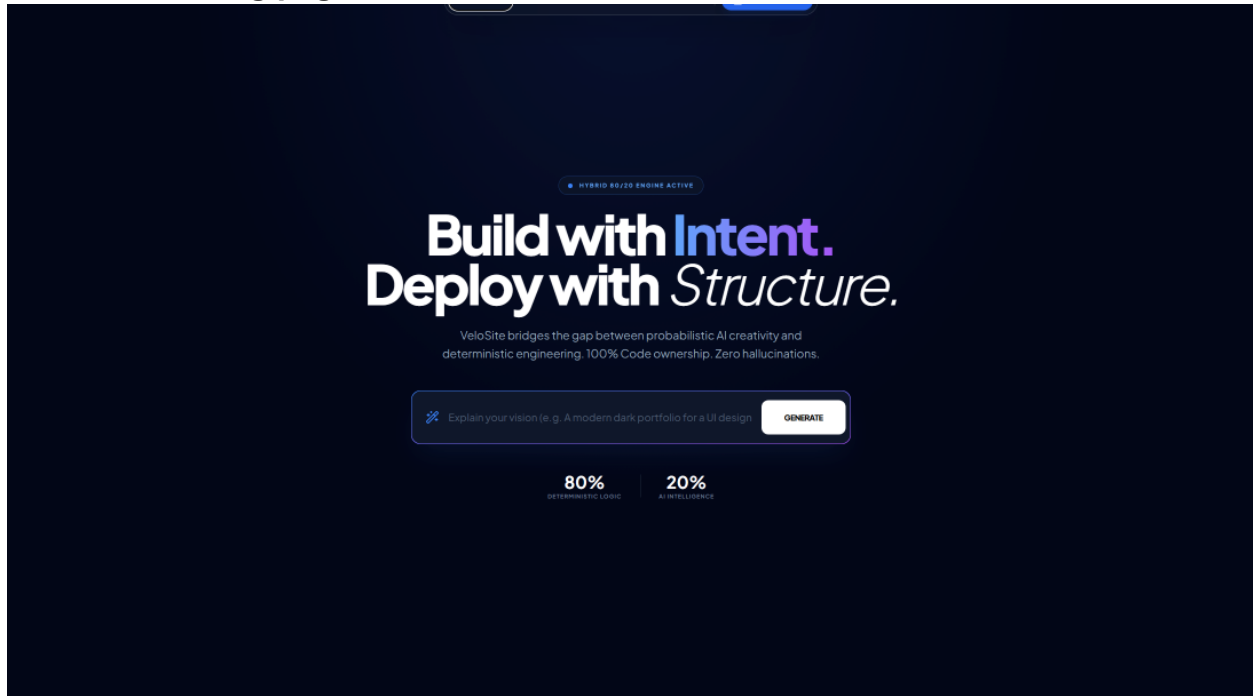


Figure 23 landing page

4.9.2. Command Centre

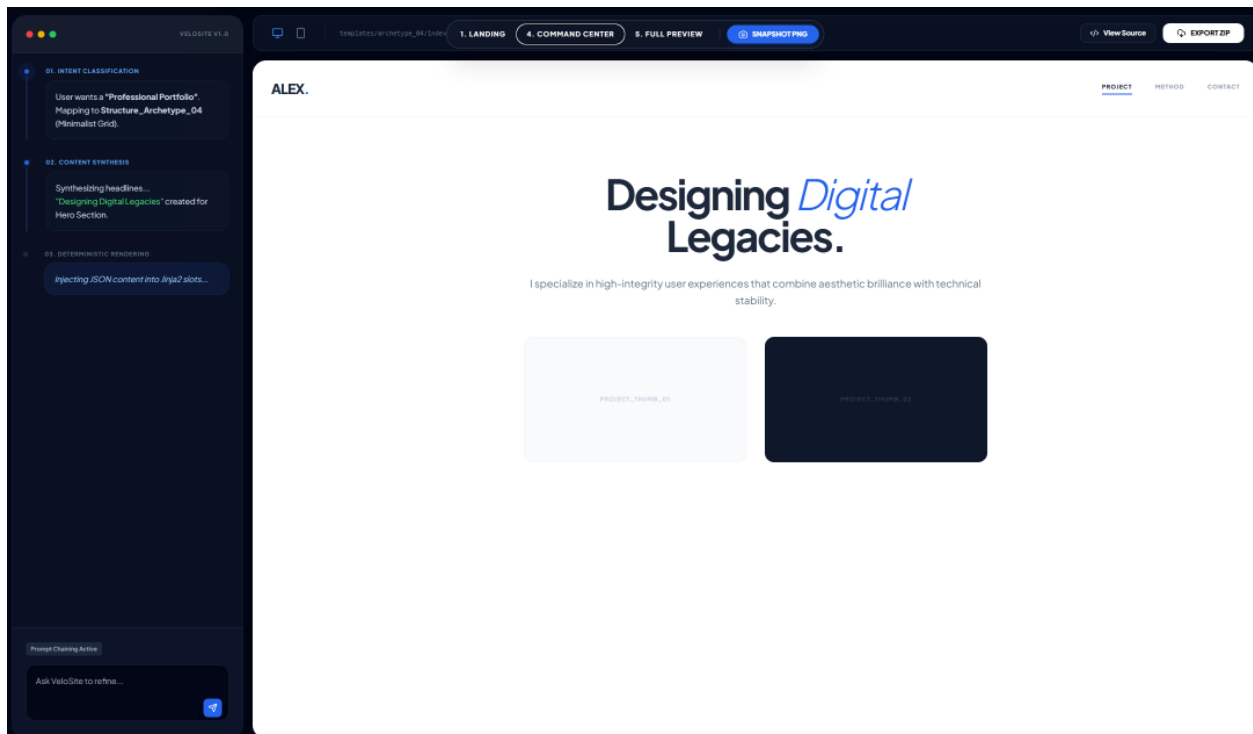


Figure 24 Command Centre page

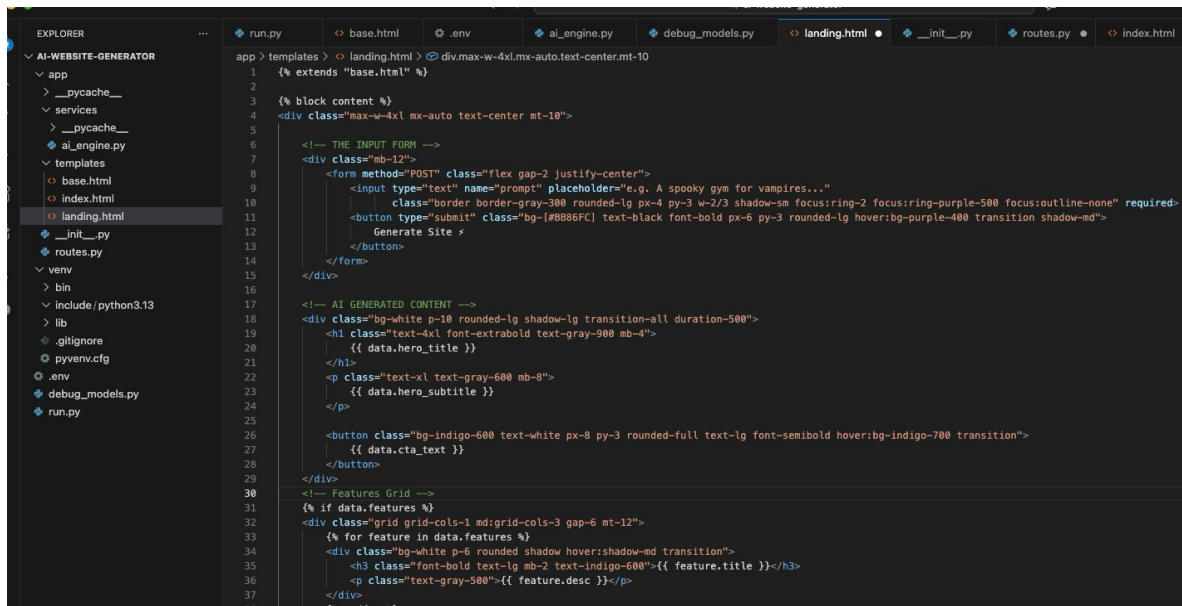
4.9.3. Full preview



Figure 25 Full preview Page

4.10. Development Status with proof

4.10.1. Setup of Frontend project Structure

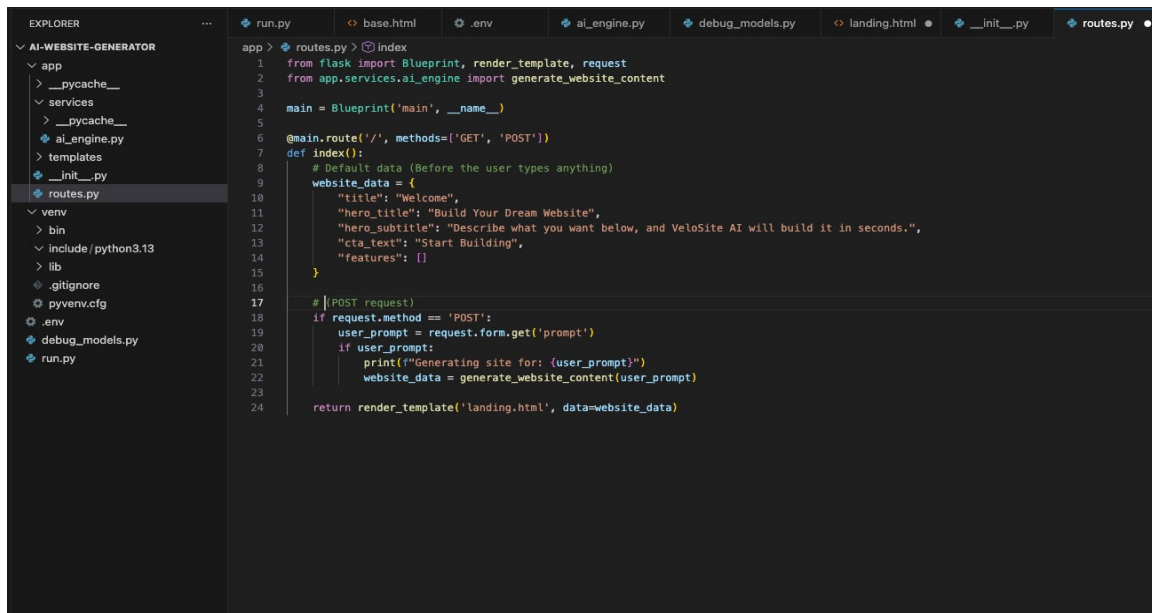


The screenshot shows the VS Code interface with the Explorer on the left and the landing.html file open in the editor. The Explorer shows the project structure for 'AI-WEBSITE-GENERATOR' with folders like 'app', 'services', 'templates', and files like 'base.html', 'index.html', 'landing.html', 'init.py', 'routes.py', and 'run.py'. The landing.html file contains HTML code for a landing page, including a form for generating a website and a grid of features.

```
1 {% extends "base.html" %}
2
3 {% block content %}
4 <div class="max-w-4xl mx-auto text-center mt-10">
5
6 <!-- THE INPUT FORM -->
7 <div class="mb-12">
8   <form method="POST" class="flex gap-2 justify-center">
9     <input type="text" name="prompt" placeholder="e.g. A spooky gym for vampires..."
10       class="border border-gray-300 rounded-lg px-4 py-3 w-2/3 shadow-sm focus:ring-2 focus:ring-purple-500 focus:outline-none" required>
11     <button type="submit" class="bg-[#808080] text-black font-bold px-6 py-3 rounded-lg hover:bg-purple-400 transition shadow-md">
12       Generate Site
13     </button>
14   </form>
15 </div>
16
17 <!-- AI GENERATED CONTENT -->
18 <div class="bg-white p-10 rounded-lg shadow-lg transition-all duration-500">
19   <h1 class="text-4xl font-extrabold text-gray-900 mb-4">
20     {{ data.hero_title }}
21   </h1>
22   <p class="text-xl text-gray-600 mb-8">
23     {{ data.hero_subtitle }}
24   </p>
25
26   <button class="bg-indigo-600 text-white px-8 py-3 rounded-full text-lg font-semibold hover:bg-indigo-700 transition">
27     {{ data.cta_text }}
28   </button>
29 </div>
30
31 <!-- Features Grid -->
32 {% if data.features %}
33 <div class="grid grid-cols-1 md:grid-cols-3 gap-6 mt-12">
34   {% for feature in data.features %}
35   <div class="bg-white p-6 rounded shadow hover:shadow-md transition">
36     <h3 class="font-bold text-lg mb-2 text-indigo-600">{{ feature.title }}</h3>
37     <p class="text-gray-500">{{ feature.desc }}</p>
38   </div>
39   </for>
40 </div>
41 </div>
```

Figure 26 Project Setup Frontend

4.10.2. Setup of Backend project Structure



The screenshot shows the VS Code interface with the Explorer on the left and the routes.py file open in the editor. The Explorer shows the project structure for 'AI-WEBSITE-GENERATOR' with folders like 'app', 'services', 'templates', and files like 'base.html', 'index.html', 'landing.html', 'init.py', 'routes.py', and 'run.py'. The routes.py file contains Python code for the Flask application, including imports, a main function, and a route for the index page.

```
1 from flask import Blueprint, render_template, request
2 from app.services.ai_engine import generate_website_content
3
4 main = Blueprint('main', __name__)
5
6 @main.route('/', methods=['GET', 'POST'])
7 def index():
8   # Default data (Before the user types anything)
9   website_data = {
10     "title": "Welcome",
11     "hero_title": "Build Your Dream Website",
12     "hero_subtitle": "Describe what you want below, and VeloSite AI will build it in seconds.",
13     "cta_text": "Start Building",
14     "features": []
15   }
16
17   # [POST request]
18   if request.method == 'POST':
19     user_prompt = request.form.get('prompt')
20     if user_prompt:
21       print(f"Generating site for: {user_prompt}")
22       website_data = generate_website_content(user_prompt)
23
24   return render_template('landing.html', data=website_data)
```

Figure 27 Project setup backend

5. Analysis of Progress

5.1. Progress Review

To date, VeloSite with its current milestone, has already completed all research, requirement analysis, and architectural design phases, thus setting a solid foundation on its blueprint of the 80/20 hybrid architecture. The current development direction is more centered on backend orchestration with the integration of the Google Gemini API (Increment 3) at 100 per cent. complete, and the Flask foundation (Increment 1) at 60 per cent. complete. This delay is consistent with an artificial iterative approach, even though the deterministic template library and validation gates have not been methodically developed yet, that will complete the probabilistic intelligence layer before it is deployed with the deterministic structural scaffolding. Having reduced the risk of integration works, which are now considered to be of high risk, the project is also in good position to start rendering and bundling processes, thus being able to meet all the technical milestones and testing conditions well before the final submission date.

5.2. Action Plan

Short term interventions will solidify the validation gate by applying the JSON schema checks and fall back dictionaries that will counter off the malformed AI responses and stabilize the system. At the same time, the frontend will be refined which will include loading indicators as well as iframe-based previews to improve the user experience. The 80/20 hybrid claim will be authenticated by undertaking a strong quality control step by carrying out unit, integration and user acceptance testing to gauge success based on Blank Canvas Paralysis that had been identified within the problem domain. Lastly, the application will be deployed to a cloud server such as Render or Heroku and all technical documentation completed to demonstrate that the application can work.

5.3. Progress Table

The Following table represents the development status of VeloSite as a of the current milestone. The project is currently in the Integration and Refining Phase.

S.N.	Tasks	Status	Progress
1	Topic Selection & Feasibility Study	Completed	100%
2	Project Proposal & Supervisor Approval	Completed	100%
3	Literature Review (AI Hallucinations & Hybrid Models)	Completed	100%
4	Requirement Analysis (SRS Document)	Completed	100%
5	System Design (DFD, Sequence, Activity Diagrams)	Completed	100%
6	Increment 1: Flask Backend & Project Foundation	On Going	60%
7	Increment 2: Jinja2 Template Library (Deterministic 80%)	Not Started	0%
8	Increment 3: Google Gemini API Integration (AI 20%)	On Going	100%
9	Increment 4: JSON Schema Validation & Fault-Tolerance	Not Started	0%
10	Increment 5: Real-time Preview & UI Polish	Not Started	0%
11	Increment 6: ZIP File Bundling & Export Logic	Not Started	0%
12	System Testing (Unit & Integration Testing)	Not Started	0%
13	User Acceptance Testing (UAT)	Not Started	0%
14	Final Report Documentation	On Going	20%
15	Project Deployment & Final Submission	Not Started	0%

Table 4 Progress Table

6. Future work

6.1. Remaining task analysis with Gantt Chart

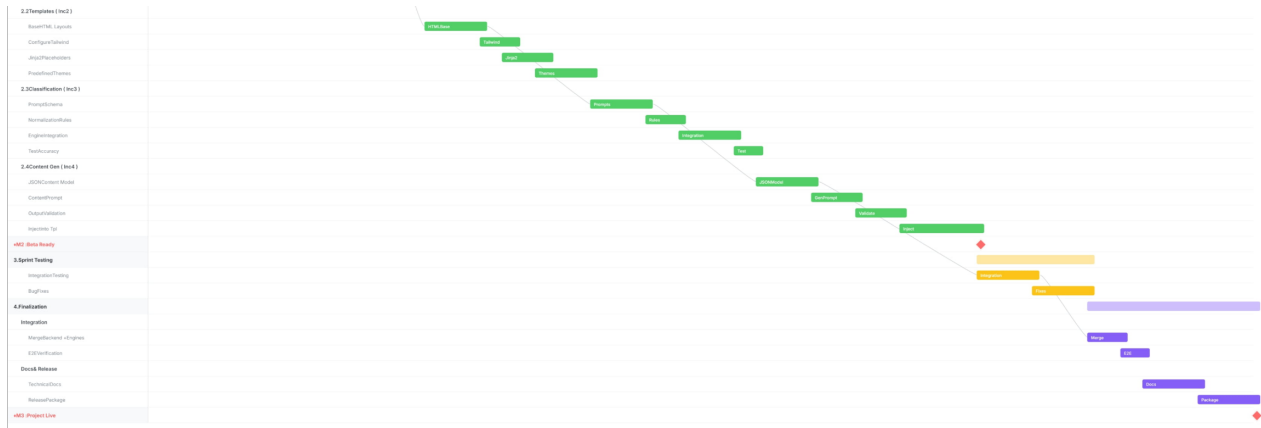


Figure 28 Remaining Task

6.2. Task Completion According to plan

VeloSite has been able to complete all research, requirement analysis, and system design goals as seen in Table thereby providing a strong base towards the 80/20 hybrid architecture. The backend core is currently focused on development and the Flask foundation (Increment 1) is at 60% and the essential Google Gemini API integration (Increment 3) is fully implemented. Even though the deterministic templates and validation gates are not listed as Started yet, this is in accordance with the deliberate Agile Incremental approach of refining the probabilistic intelligence layer prior to finishing off the structural scaffolding. Through the high-risk integration tasks that were overcome, the project is expected to start rendering and bundling increments and maintain adequate momentum to finish all testing and documentation by the endpoint of the final submission date.

7. Conclusion

To conclude, the creation of VeloSite has made a crucial step forward and the main architectural hypothesis, namely the 80/20 Hybrid Model, is finally confirmed. The effort has gone past the basic stages of research, requirement analysis and system design and has thus developed a strong technical base. So far, the most interesting accomplishments include introduction of Flask backend orchestration in Increment 1 and the high-fidelity integration of the Google Gemini API in Increment 3. These achievements show how well the system can analyze natural language and build structured data in the form of a JSON to successfully mediate between the abstract intentions of the user and concrete digital information.

The final development stage will now switch to the deterministic elements necessary to ensure full structural integrity and fault tolerance. The key technical goals that it is still pursuing include the finishing of the Jinja2 and the Tailwind template library in Increment 2 and the hardening of the JSON-schema validation gate in Increment 4. After completion of rendering and ZIP-export logic, a strict quality-assuring cycle will be engaged in the project. This cycle will entail unit and integration testing, and User Acceptance Testing (UAT), to ensure that the platform is successfully removing AI hallucinations and provides a viable Prompt-to-Deploy workflow to non-technical users.

So far, the evolutionary path of VeloSite has been an extreme exploration of how far to bring the creative freedom of probabilistic AI against the strict discipline of traditional software engineering. The exploration of the complexities of immediate engineering and JSON verification has provided invaluable information into the development of sandboxed AI environments with a strong focus on system stability. Even though the optimization of the fault-tolerant systems required a great deal of experimentation, the obtained results inspire a lot of confidence during the project. VeloSite still is on track to overcome the Blank Canvas Paralysis affecting modern web development, as well as to meet all the academic requirements before the deadline of December 2025.

8. References

Rokis, K. a. K. M., 2023. *Exploring Low-Code Development: A Comprehensive Literature Review*. [Online]

Available at: [https://csimq-journals.rtu.lv/\(Anon., n.d.\)/view/csimq.2023-36.04](https://csimq-journals.rtu.lv/(Anon., n.d.)/view/csimq.2023-36.04)
[Accessed 21 December 2025].

Pearce, H. A. B. T. B. D.-G. B. a. K. R., 2022. *Asleep at the Keyboard? Assessing the Security of GitHub Copilot's Code Contributions*. [Online]

Available at: <https://ieeexplore.ieee.org/document/9833571>
[Accessed 21 December 2025].

Kabir, S. U.-I. D. K. B. a. Z. T., 2024. *Who Answers It Better? An In-Depth Analysis of ChatGPT and Stack Overflow Answers to Software Engineering Questions*.

[Online] Available at: <https://link.springer.com/article/10.1007/s10664-023-10421-2>
[Accessed 21 December 2025].

White, J. F. Q. H. S. S. M. O. C. G. H. E. A. a. S. D., 2023. *White, J., Fu, Q., Hays, S., Sandborn,*

M., Olea, C., Gilbert, H., Elnawawy, A. and Schmidt, D.C..

[Online] Available at: <https://arxiv.org/abs/2302.11382>
[Accessed 22 December 2025].

Wix.com, 2025. *AI website Builder-build a Website with AI*.

[Online] Available at: <https://www.wix.com/ai-website-builder>
[Accessed 22 December 2025].

Vercel, 2024. *v0.dev - Generative UI*. [Online]

Available at: <https://v0.dev>
[Accessed 21 Decmber 2025].

Dias, C., 2023. *Visual Studio Code and GitHub Copilot*. [Online]

Available at: <https://code.visualstudio.com/blogs/2023/03/30/vscode-copilot>
[Accessed 24 December 2025].

Day, B., 2025. *What Is The Waterfall Methodology?*. [Online]

Available at: <https://www.forbes.com/advisor/business/what-is-waterfall-methodology/>
[Accessed 21 December 2025].

Paliwal, P., 2025. *What is Agile Methodology?*. [Online]
Available at: <https://www.geeksforgeeks.org/software-testing/what-is-agile-methodology/>
[Accessed 21 December 2025].

Anon., n.d. [Online].

Casucian, I., 2024. *What Is Waterfall Project Management?*. [Online]
Available at: <https://technologyadvice.com/blog/project-management/what-is-waterfallproject-management/> [Accessed 22 December 2025].

Visaul Paradigm, 2025. *What is Sequence Diagram?*. [Online]
Available at: <https://www.visual-paradigm.com/guide/uml-unified-modelinglanguage/what-is-sequence-diagram/> [Accessed 24 December 2025].

9. Appendix

9.1. Introduction

9.1.1. Purpose

This document aims at giving a specification to the VeloSite system. It stipulates the non-functional and functional specifications of the Intent-to-Structure hybrid website generator. The paper will focus on the project managers, independent auditors, and prospective programmers.

9.1.2. Document Conventions

This paper is based on the IEEE 830-1998 standard. The requirements are assigned with the label Must (Essential), Should (Recommended), or Could (Optional).

9.1.3. Audience and Reading recommendations.

Supervisors/Examiners: Pay attention to the Section 2.1 (Perspective) and the Section 5.1 (Fault Tolerance) in order to find out the unique worth of the project.

Developers: Implementation details: Sec. 3 (Interfaces): Section 4 (System Features).

9.1.4. Product Scope

VeloSite is a web application that is fault tolerant and is used to remove the hallucinations that can happen in web development that are referred to as AI hallucinations. VeloSite is an 80/20 Hybrid unlike raw Text-to-Code generators. It is creative intent and content (20%) based on Google Gemini API and transfers the logical structure to a deterministic Flask/Jinja2 engine (80%).

9.1.5. References

Pearce, H. (2022). Hallucinations in LLM Code Generation.

Google Gemini API Documentation.

Jinja2 Documentation of the Templating Engine.

9.2. Overall Description

9.2.1 Product Perspective

VeloSite is an infrastructure tool that is a standalone web-based one. It serves as an abstraction layer between the User and the Gemini API so that any AI-generated data can always be enclosed in valid and responsive engineering templates.

9.2.2 Product Functions

Interpretation of prompts in natural language.

Automated design intent (Portfolio vs. Blog vs. Landing Page).

Content generation by AI (Headlines, text of the body, slogans).

Deterministic conversion to HTML5/Tailwind CSS.

Exporting of one-click ZIP source code.

9.2.3 Characteristics and Classes of users.

Non-Technical Prosumers: The end users that are code-illiterate and need professional level websites.

Rapid Prototypers: human Developers that require a boilerplate structure which is created on the fly and can be refined.

9.2.4. Operating Environment

Client Side: Newer browsers (Chrome, Firefox, Safari).

Server Side: Python 3.12, Flask Framework.

API: Google Gemini Pro/Flash API.

9.2.5. Constraints of the Design and Implementation.

API Latency: The system will rely on the server response times in Google server.

Statelessness: There is no database being used; all project states are momentarily just until the time of export.

The assumptions and dependencies are to be stated in 2.6.

Assumption: Prompts will be given in English by the users.

Dependency: API interaction is dependent on constant internet connection.

9.3. External Interface Requirement.

9.3.1. User Interfaces

A Command Center dashboard comprising of a prompt input field, a live preview-iframe window and a main download action button.

9.3.2 Hardware Interfaces

The system should be user-friendly and workable on gadgets with the lowest resolution of 375px (Mobile) and the highest of 1920px (Desktop).

9.3.3 Software Interfaces

Google Gemini API: Structured JSON Mode.

Jinja2 Engine: Local use.

Tailwind CSS: Cdn based styling of preview.

9.3.4 Communications Interfaces

Any information that is transmitted between User and Flask Backend and Gemini API should be done over HTTPS (port 443).

9.4. System Features

9.4.1 Feature 1 Classifying the intent (AI Component):

Priority: High

Description The system examines the user prompt to select a template archetype.

REQ-1: The system should be able to recognize the industry (e.g. Photography, Tech) based on the prompt.

REQ-2: The system should give a particular template ID (e.g. theme01).

9.4.2 Feature 2 Content Generation (AI Component).

Priority: High

Description: Gemini will produce the particular text required in every part of the site.

REQ-1: The system should produce a JSON with the herotitle, abouttext and servicedescriptions.

9.4.3 Feature 3 Deterministic Rendering (System Component).

Priority: High

Description The Flask application infuses AI information into Jinja2 templates.

REQ-1: The system should be able to check the JSON schema and render it.

REQ-2: In case the response of the AI is malformed, the system should fallback to the "Default Assets".

9.5. Other Nonfunctional Requirements.

9.5.1 Performance Requirements

The system should not take longer than 25 seconds to complete the generation lifecycle (Prompt to Preview).

9.5.2 Safety Requirements

The system should be used to purify any user input to avoid Cross-Site Scripting (XSS) or injection attacks.

9.5.3 Security Requirements

The API Keys should be kept in the environment variables (.env) and can not be displayed to the client-side JavaScript.

9.5.4 Software Quality Attributes (Fault Tolerance).

Integrity: The created site structure has to be 100 percent valid HTML/CSS. The deterministic engine should strip the AI during validation in a non-successful re-creation of a given tag that the AI hallucinates.

9.5.5 Business Rules

Code Ownership: The resultant output should be open-source files that should not be vendor locked.

9.6. Appendix A: Glossary

Hallucination: A syntactically correct code generator with syntactically invalid code generation.

Deterministic A process that gives the same output given the same input, and is not random.

9.7. Description on Methodology and its phases

The development is divided into three phases:

Phase 1: Planning & Design

This stage takes linear form in order to establish the backbone of the project.

Requirements Analysis: Establishment of the overall list of system requirements, including the architectural constraints of the 80/20 types.

System Design: Specification of the top-level architecture, theme frameworks, and how modules interrelate with one another to provide consistency in the future increments.

Phase 2: Repeat Development Cycles (The Increments)

The main implementation is split into four distinct increments by the implementation loop with each increment being topped by a Sprint Testing and Review loop to ensure that everything is stable before the next increment.

- **Increment 1 (Backend Foundation):** The Flask server is developed, the routes are configured, the input validation, and the configuration are managed.

- **Increment 2 (Template and Theme Engine):** The deterministic site structure is built with the help of HTML5, Tailwind CSS, and Jinja2 placeholders.
- **Increment 3 (Classification Engine):** Incorporation of AI logic to profile ambiguous user input prompts to precise templates and themes.
- **Increment 4 (Content Generation Engine):** Integration of the Generative AI module to generate textual content (headlines, descriptions) in form of structured JSON, based on the context.

Phase 3 Finalisation and Deployment.

The final stage is about system integration which involves bringing together all the modules to make the whole workflow come to life. This is then accompanied by intense testing, cross-device validation and performance evaluation and finally the web server is rolled out.

9.8. Data Flow Diagram Detail level explanation

The Level 0 Data Flow Diagram (Context Diagram) is used to show the scope and boundary of the VeloSite system. It provides two external parties: the User and the Google Gemini API. The main data flow is the User entering a prompt in natural language and in exchange obtain real-time preview of a web site and a downloadable package of sourcecode. That the AI is an external service is highlighted in the diagram, which only communicates with the system through organized data communications.

The Level 1 DFD divides the system into five major processes in order to demonstrate the 80/20 hybrid architecture. Processes 2.0 (Intent Classification) and 3.0 (Content Generation) are located in an A.I. Components boundary, the 20 percent that is stochastic. These procedures communicate the structured JSON data with Google Gemini API. On the other hand, Processes 1.0, 4.0, and 5.0 are deterministic (the rest 80 percent). Process 4.0 (Jinja2 Template Rendering) is worth noting, since, by

accessing templates stored in the 'Template Library (D1), templates are generated outside the HTML structure, thus allowing the AI to perform other functions without direct editing of the HTML structure to reduce the risk of structural hallucinations.

The Level 2 DFD supplies a deep break down of Process 4.0 (Jinja2 Template Rendering). This step is central to the so-called fault-tolerance of the system. Process 4.1 (JSON Parser and Schema Validator) is a security gate that ensures that all data sent to the AI components is syntactically accurate before it is sent to the Template Path Selector (4.2). The system separates data mapping (4.3) and environment loading (4.4), which ensures that the result is a high-quality responsive site that meets pre-validated engineering requirements.

9.9. Survey response the project

1. How would you rate your current web development skills?

61 responses

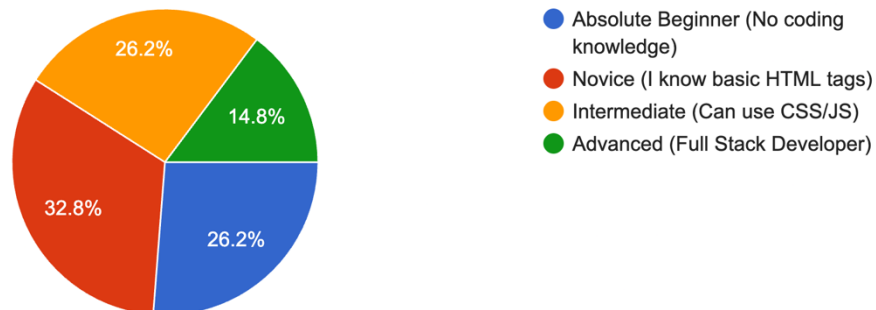


Figure 29 Survey question 1

2. How often do you need to create a website or landing page?

61 responses

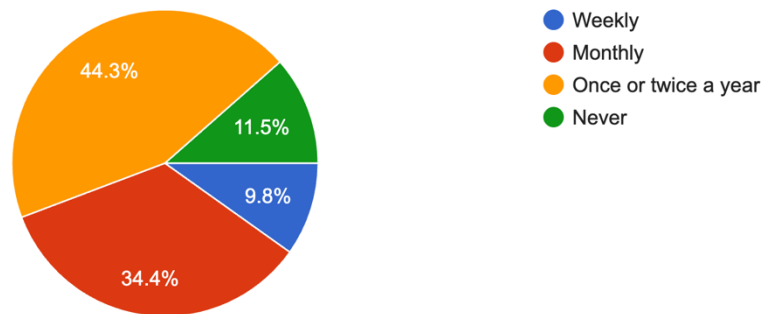


Figure 30 Survey Question 2

3. On average, how long does it take you to build a responsive, good-looking landing page from scratch?

61 responses

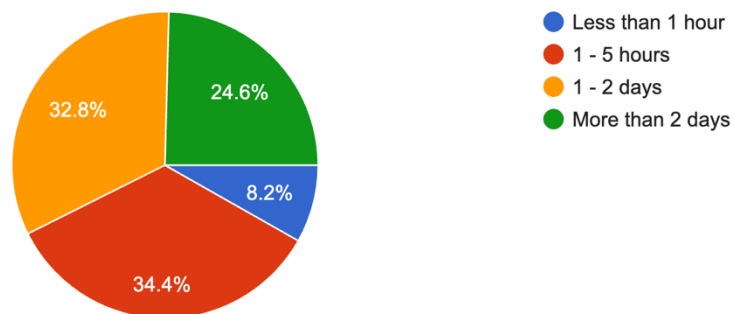


Figure 31 Survey question 3

4. What do you find most frustrating about building websites manually? (Select multiple)

61 responses

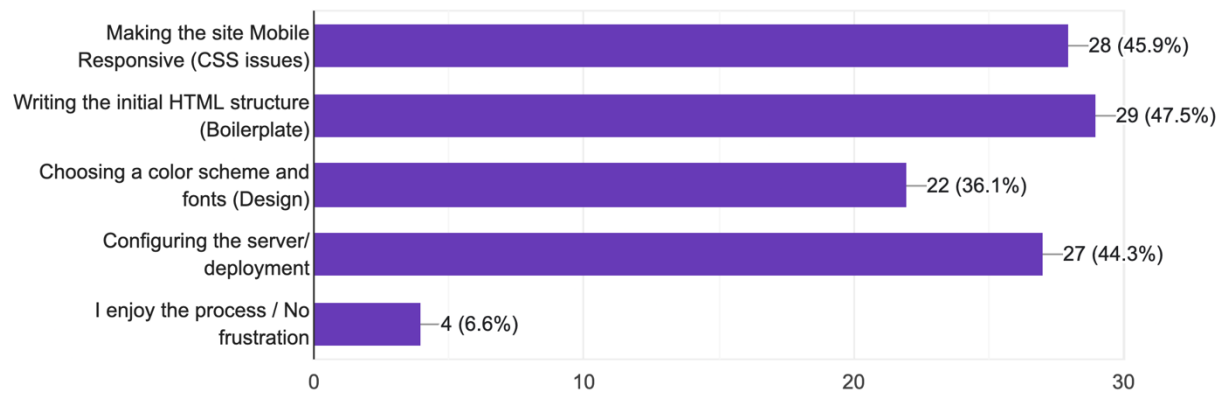


Figure 32 Survey Question 4

5. Do you ever experience "Blank Canvas Paralysis" (not knowing where to start) when designing a UI?

61 responses

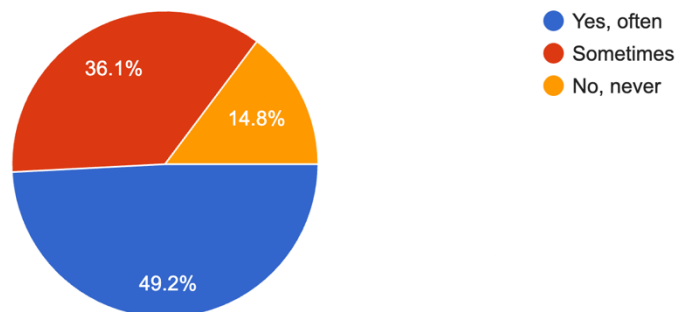


Figure 33 Survey Question 5

6. Have you ever used Generative AI tools (ChatGPT, Gemini, Copilot) to help you write code?

61 responses

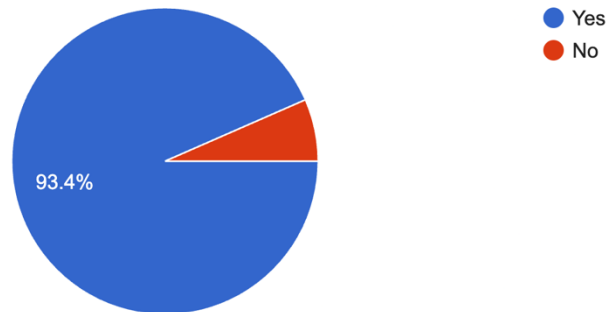


Figure 34 Survey Question 6

8. When using AI to generate code, how often do you have to manually fix bugs in the output?

60 responses

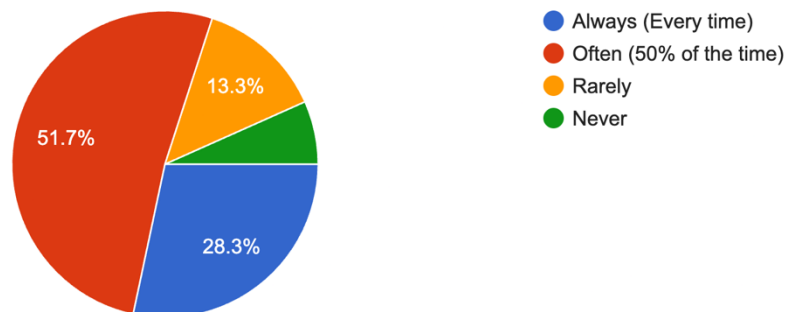


Figure 35 Survey Question 7

8. When using AI to generate code, how often do you have to manually fix bugs in the output?

60 responses

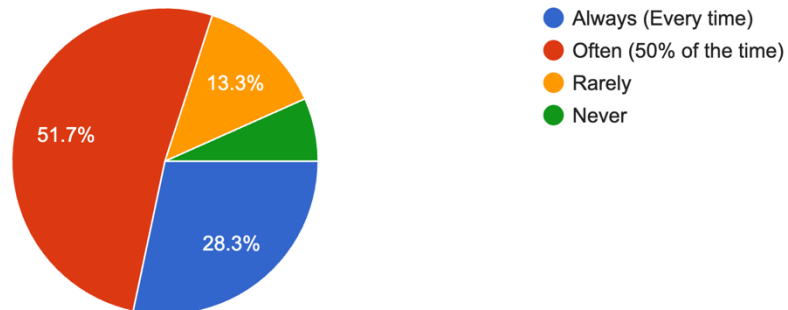


Figure 36 Survey question 8

9. How important is it for you to have "Code Ownership" (the ability to download and host the source code yourself)?

60 responses

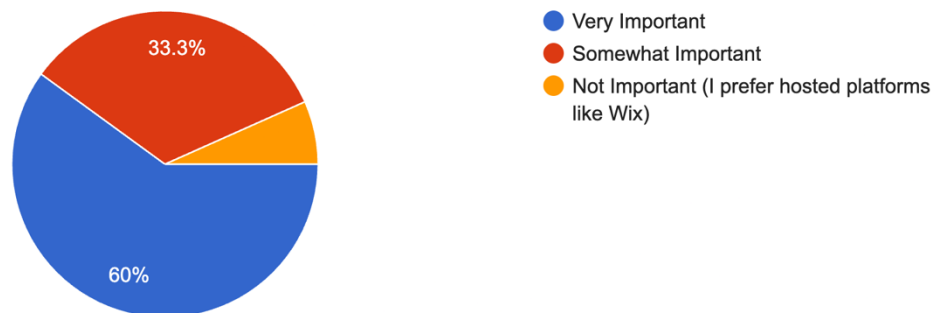


Figure 37 Survey question 9

10. Would you prefer a "Text-to-Website" tool over a "Drag-and-Drop" editor (like Wix) if it was faster?

60 responses

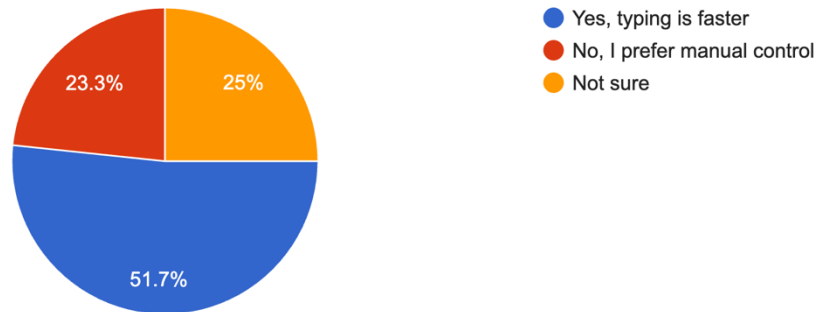


Figure 38 Survey question 10

11. If a tool could generate a fully responsive, bug-free website from a single sentence in 30 seconds, would you use it?

60 responses

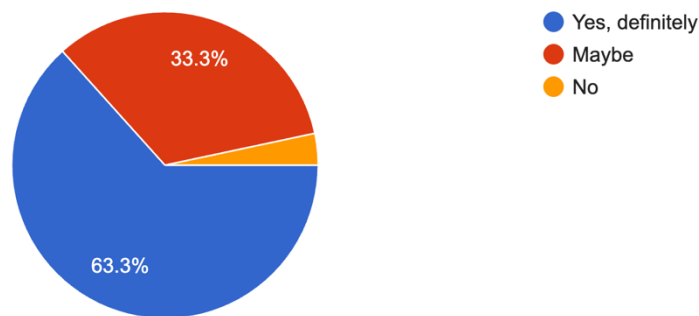


Figure 39 Survey question 11

12. Which feature would be most valuable to you?

60 responses

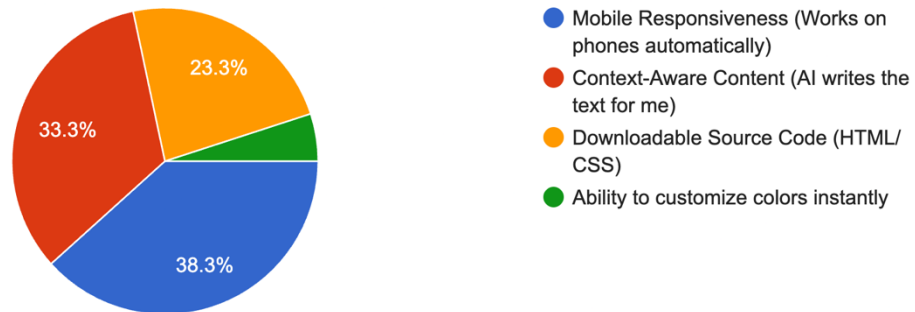


Figure 40 Survey Question 12

13. Would you find a feature that generates a website based on an uploaded image (sketch/screenshot) useful?

60 responses

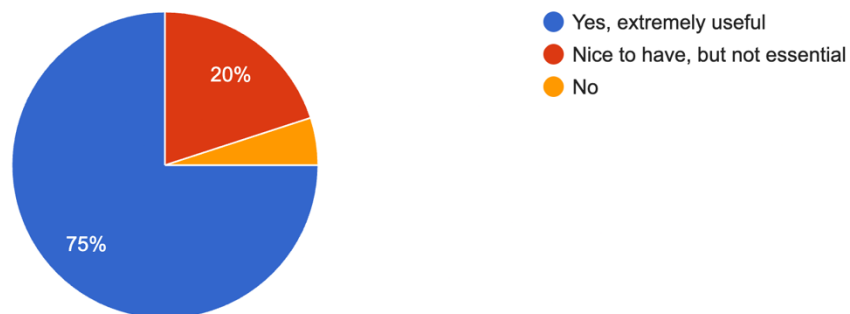


Figure 41 Survey Question 13xx

14. Are you willing to use a "Template-Based" AI (fixed layout but reliable) vs. a "Free-Form" AI (might break)?

60 responses

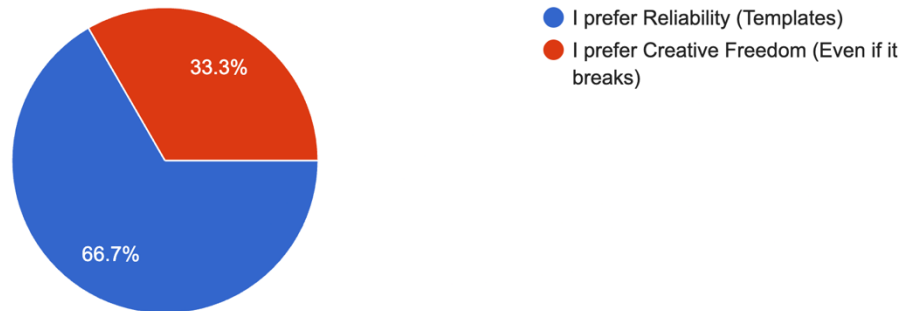


Figure 42 Survey Question 14